Digital Systems Project Assignment 2
Digital Watch: Stopwatch and Timer

This is the second of two assignments. Building on your settable watch from Assignment 1, you will add a stopwatch, a timer, and a display brightness control. You will also be introduced to formal verification and see first-hand how a synthesiser translates your code into actual hardware. Greater emphasis is placed on explaining the design decisions along the way.

1. Policies and Assessment

These are the same as for Assignment 1.

Back up your work regularly.

2. Automated Tests

The automated test suite, accessed by running `pytest` in the terminal, includes per-module functional tests, the formal verification tests introduced later in this assignment, and checks that no module has linting or compilation warnings or errors.

Run `pytest` now to confirm that all your files from Assignment 1 are in order before proceeding.

For the modules written as part of this assignment, some will have formal verification tests only, some will have cocotb tests only, and some will have both. Formal verification is not a panacea and is generally not the right tool to use for testing responses to sequences of inputs.

```
(venv) root@4034a70d84cc:/workspace# pytest
============================= test session starts ==============================
platform linux -- Python 3.12.3, pytest-9.0.3, pluggy-1.6.0
rootdir: /workspace
configfile: .pytest.toml
testpaths: tests
plugins: metadata-3.1.1, html-4.2.0
collected 112 items

tests/test_arming_latch.py .                                             [  0%]
tests/test_binary_to_bcd.py .                                            [  1%]
tests/test_button_auto_repeat.py .                                       [  2%]
tests/test_button_hold_detect.py .                                       [  3%]
tests/test_button_hold_pulse.py .                                        [  4%]
tests/test_decimal_display_driver.py .                                   [  5%]
tests/test_edit_mode_selector.py .                                       [  6%]
tests/test_editable_counter.py ...                                       [  8%]
tests/test_formal.py ssssssss                                            [ 16%]
tests/test_hms_counter.py ...                                            [ 18%]
tests/test_key_synchroniser.py .                                         [ 19%]
tests/test_lint.py ..........................................            [ 60%]
tests/test_mod_n_counter.py ..                                           [ 62%]
tests/test_pwm_generator.py .......                                      [ 68%]
tests/test_restartable_rate_generator.py ......                          [ 74%]
tests/test_rising_edge_detector.py .                                     [ 75%]
tests/test_seven_segment.py ..                                           [ 76%]
tests/test_stopwatch_control.py s                                        [ 77%]
tests/test_top_de1_soc.py .                                              [ 78%]
tests/test_top_time_display_v1.py .                                      [ 79%]
tests/test_up_down_counter.py ...                                        [ 82%]
tests/test_user_top_brightness_timepiece.py s                            [ 83%]
tests/test_user_top_brightness_wrapper.py ssss                           [ 86%]
tests/test_user_top_stopwatch_v1.py s                                    [ 87%]
tests/test_user_top_timepiece_v1.py s                                    [ 88%]
tests/test_user_top_timer_v1.py s                                        [ 89%]
tests/test_user_top_watch_v1.py .                                        [ 90%]
tests/test_user_top_watch_v2.py ..                                       [ 91%]
tests/test_user_top_watch_v3.py ....                                     [ 95%]
tests/test_user_top_watch_v4.py .....                                    [100%]

======================== 95 passed, 17 skipped in 22.01s ========================
(venv) root@4034a70d84cc:/workspace#
```

3. Formal Verification

Formal verification is introduced here using an up-down counter with reset as the running example.

3.1. Up-Down Counter with Reset

The up-down counter from Assignment 1 lacked reset capability.

> **Report 1**
>
> When a module needs to be extended mid-design, several approaches are possible: modify the original in place; copy it, modify the copy, and leave the original untouched; or copy it, modify the copy, and rewrite the original as a thin wrapper around the new module. Can you think of any other options? For this specific case, which approach would you take, and how would you name the resulting module or modules? Justify your choices and discuss their trade-offs relative to the alternatives.

SystemVerilog's support for default port values makes modifying the original module particularly attractive here. Adding `input logic rst = 1'b0` to the port list means existing instantiations that do not connect `rst` continue to work unchanged, while new instantiations can drive it explicitly. Any regression introduced during the modification would be caught by the existing test suite; the suite should then be extended to cover the new reset behaviour.

In practice, however, Verilator and Icarus Verilog both lack support for default port values; we therefore leave the existing module untouched and create a new one from a copy.

Task Implement the following module in `rtl/up_down_counter_rst.sv`, following all conventions from Assignment 1, including **elimination of all Verilator linting warnings.**

rtl > up_down_counter_rst.sv > ...

```sv
// Up_Down_Counter_Rst
// To act as a standard Counter, whose modulus value is determined by the
// MAX parameter argument that was passed into it. The counter will count up
// when the up input is high, and count down when the up input is low. The
// counter will wrap back to 0 when the maximum value is reached, and wrap
// back to the maximum value when the minimum value is reached for up and
// down counting, respectively.
// The input `rst` signal allows asynchronous reseting of the `count` output
//
// Parameters:
//   MAX    - Maximum count value (default: 2).
//   WIDTH  - Width of the count output (default: 2).
//
// Ports:
//   clk    - Clock input.
//   rst    - When high, counter becomes and remains 'd0 on every rising edge
//   enable - When high, the counter is enabled.
//   up     - When high, the counter counts up; when low, counts down.
//   count  - Output Signal, returns [WIDTH-1:0] number between 0 and N-1
//
// Local Variables:
//   [WIDTH-1:0] Max:        - Casts `MAX` parameter to local `Max` parameter
//   [WIDTH-1:0] next_count: - Intermediary variable to hold value of `count`'s
//                             next number

`timescale 1ns / 1ps

module up_down_counter_rst #(
    parameter int MAX   = 2,
    parameter int WIDTH = 2
) (
    input logic clk,
    input logic rst,
    input logic enable,
    input logic up,
    output logic [WIDTH-1:0] count = WIDTH'(0)
);

  // Declare local parameters and signals
  // Cast input `MAX` parameter to local `Max` parameter
  localparam logic [WIDTH-1:0] Max = WIDTH'(MAX);
  logic [WIDTH-1:0] next_count;

  // State, and Output Logic
  always_ff @(posedge clk) begin
    if (rst) count <= WIDTH'(0);
    else if (enable) count <= next_count;
  end

  // Next state logic
  always_comb begin
    if (up) begin  // up=1 -> Count up
      if (count < Max) next_count = count + WIDTH'(1);
      else next_count = WIDTH'(0);  // Wrap to 0 when Max
    end else begin  // else up=0 -> Count down
      if (count > WIDTH'(0)) next_count = count - WIDTH'(1);
      else next_count = Max;  // Wrap to Max when 0
    end
  end

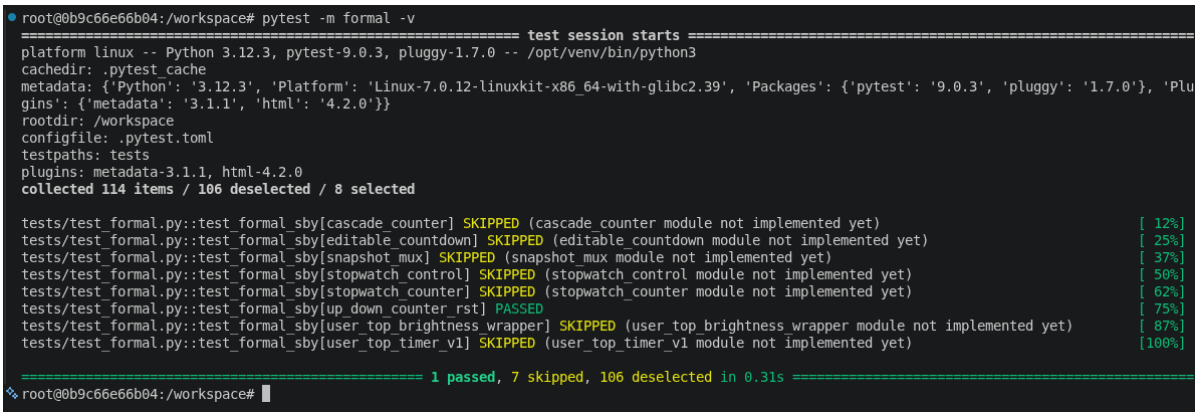
endmodule
```

```
(venv) root@4034a70d84cc:/workspace# verilator -Wall --lint-only -y rtl rtl/up_down_counter_rst.sv
(venv) root@4034a70d84cc:/workspace# iverilog -Wall -g2012 -Y .sv -y rtl rtl/up_down_counter_rst.sv
(venv) root@4034a70d84cc:/workspace#
```

```
root@0b9c66e66b04:/workspace# sby -f sby/up_down_counter_rst.sby
SBY  5:50:35 [sby/up_down_counter_rst_max_4] Removing directory '/workspace/sby/up_down_counter_rst_max_4'.
SBY  5:50:35 [sby/up_down_counter_rst_max_4] Copy '/workspace/rtl/up_down_counter_rst.sv' to '/workspace/sby/up_down_counter_rst_max_4/src/up_down_counter_rst.sv'.
SBY  5:50:35 [sby/up_down_counter_rst_max_4] Copy '/workspace/sby/up_down_counter_rst_assert.sv' to '/workspace/sby/up_down_counter_rst_max_4/src/up_down_counter_rst_assert.sv'.
SBY  5:50:35 [sby/up_down_counter_rst_max_4] engine_0: smtbmc z3
SBY  5:50:35 [sby/up_down_counter_rst_max_15] Removing directory '/workspace/sby/up_down_counter_rst_max_15'.
SBY  5:50:35 [sby/up_down_counter_rst_max_15] Copy '/workspace/rtl/up_down_counter_rst.sv' to '/workspace/sby/up_down_counter_rst_max_15/src/up_down_counter_rst.sv'.
SBY  5:50:35 [sby/up_down_counter_rst_max_15] Copy '/workspace/sby/up_down_counter_rst_assert.sv' to '/workspace/sby/up_down_counter_rst_max_15/src/up_down_counter_rst_assert.sv'.
SBY  5:50:35 [sby/up_down_counter_rst_max_15] engine_0: smtbmc z3
SBY  5:50:35 [sby/up_down_counter_rst_max_60] Removing directory '/workspace/sby/up_down_counter_rst_max_60'.
SBY  5:50:35 [sby/up_down_counter_rst_max_60] Copy '/workspace/rtl/up_down_counter_rst.sv' to '/workspace/sby/up_down_counter_rst_max_60/src/up_down_counter_rst.sv'.
SBY  5:50:35 [sby/up_down_counter_rst_max_60] Copy '/workspace/sby/up_down_counter_rst_assert.sv' to '/workspace/sby/up_down_counter_rst_max_60/src/up_down_counter_rst_assert.sv'.
SBY  5:50:35 [sby/up_down_counter_rst_max_60] engine_0: smtbmc z3
SBY  5:50:35 [sby/up_down_counter_rst_max_4] base: starting process "cd sby/up_down_counter_rst_max_4/src; yosys -ql ../model/design.log ../model/design.ys"
SBY  5:50:35 [sby/up_down_counter_rst_max_15] base: starting process "cd sby/up_down_counter_rst_max_15/src; yosys -ql ../model/design.log ../model/design.ys"
SBY  5:50:35 [sby/up_down_counter_rst_max_60] base: starting process "cd sby/up_down_counter_rst_max_60/src; yosys -ql ../model/design.log ../model/design.ys"
SBY  5:50:35 [sby/up_down_counter_rst_max_4] base: finished (returncode=0)
SBY  5:50:35 [sby/up_down_counter_rst_max_15] base: finished (returncode=0)
SBY  5:50:35 [sby/up_down_counter_rst_max_4] prep: starting process "cd sby/up_down_counter_rst_max_4/model; yosys -ql design_prep.log design_prep.ys"
SBY  5:50:35 [sby/up_down_counter_rst_max_60] base: finished (returncode=0)
SBY  5:50:35 [sby/up_down_counter_rst_max_15] prep: starting process "cd sby/up_down_counter_rst_max_15/model; yosys -ql design_prep.log design_prep.ys"
SBY  5:50:35 [sby/up_down_counter_rst_max_60] prep: starting process "cd sby/up_down_counter_rst_max_60/model; yosys -ql design_prep.log design_prep.ys"
SBY  5:50:35 [sby/up_down_counter_rst_max_4] prep: finished (returncode=0)
SBY  5:50:35 [sby/up_down_counter_rst_max_4] smt2: starting process "cd sby/up_down_counter_rst_max_4/model; yosys -ql design_smt2.log design_smt2.ys"
SBY  5:50:35 [sby/up_down_counter_rst_max_15] prep: finished (returncode=0)
SBY  5:50:35 [sby/up_down_counter_rst_max_15] smt2: starting process "cd sby/up_down_counter_rst_max_15/model; yosys -ql design_smt2.log design_smt2.ys"
SBY  5:50:35 [sby/up_down_counter_rst_max_60] prep: finished (returncode=0)
SBY  5:50:35 [sby/up_down_counter_rst_max_60] smt2: starting process "cd sby/up_down_counter_rst_max_60/model; yosys -ql design_smt2.log design_smt2.ys"
SBY  5:50:35 [sby/up_down_counter_rst_max_4] smt2: finished (returncode=0)
SBY  5:50:35 [sby/up_down_counter_rst_max_4] engine_0.basecase: starting process "cd sby/up_down_counter_rst_max_4; yosys-smtbmc -s z3 --presat --noprogress -t 3  --append 0 --dump-vcd
 engine_0/trace.vcd --dump-yw engine_0/trace.yw --dump-vlogtb engine_0/trace_tb.v --dump-smtc engine_0/trace.smtc model/design_smt2.smt2"
SBY  5:50:35 [sby/up_down_counter_rst_max_15] smt2: finished (returncode=0)
SBY  5:50:35 [sby/up_down_counter_rst_max_4] engine_0.induction: starting process "cd sby/up_down_counter_rst_max_4; yosys-smtbmc -s z3 --presat -i --noprogress -t 3  --append 0 --dump
-vcd engine_0/trace_induct.vcd --dump-yw engine_0/trace_induct.yw --dump-vlogtb engine_0/trace_induct_tb.v --dump-smtc engine_0/trace_induct.smtc model/design_smt2.smt2"
SBY  5:50:35 [sby/up_down_counter_rst_max_60] smt2: finished (returncode=0)
SBY  5:50:35 [sby/up_down_counter_rst_max_15] engine_0.basecase: starting process "cd sby/up_down_counter_rst_max_15; yosys-smtbmc -s z3 --presat --noprogress -t 3  --append 0 --dump-v
cd engine_0/trace.vcd --dump-yw engine_0/trace.yw --dump-vlogtb engine_0/trace_tb.v --dump-smtc engine_0/trace.smtc model/design_smt2.smt2"
SBY  5:50:35 [sby/up_down_counter_rst_max_15] engine_0.induction: starting process "cd sby/up_down_counter_rst_max_15; yosys-smtbmc -s z3 --presat -i --noprogress -t 3  --append 0 --du
mp-vcd engine_0/trace_induct.vcd --dump-yw engine_0/trace_induct.yw --dump-vlogtb engine_0/trace_induct_tb.v --dump-smtc engine_0/trace_induct.smtc model/design_smt2.smt2"
SBY  5:50:35 [sby/up_down_counter_rst_max_60] engine_0.induction: starting process "cd sby/up_down_counter_rst_max_60; yosys-smtbmc -s z3 --presat -i --noprogress -t 3  --append 0 --du
mp-vcd engine_0/trace_induct.vcd --dump-yw engine_0/trace_induct.yw --dump-vlogtb engine_0/trace_induct_tb.v --dump-smtc engine_0/trace_induct.smtc model/design_smt2.smt2"
SBY  5:50:35 [sby/up_down_counter_rst_max_4] engine_0.basecase: ##   0:00:00  Solver: z3
SBY  5:50:35 [sby/up_down_counter_rst_max_60] engine_0.basecase: starting process "cd sby/up_down_counter_rst_max_60; yosys-smtbmc -s z3 --presat --noprogress -t 3  --append 0 --dump-v
cd engine_0/trace.vcd --dump-yw engine_0/trace.yw --dump-vlogtb engine_0/trace_tb.v --dump-smtc engine_0/trace.smtc model/design_smt2.smt2"
SBY  5:50:35 [sby/up_down_counter_rst_max_4] engine_0.basecase: ##   0:00:00  Checking assumptions in step 0..
SBY  5:50:35 [sby/up_down_counter_rst_max_4] engine_0.induction: ##   0:00:00  Solver: z3
SBY  5:50:35 [sby/up_down_counter_rst_max_15] engine_0.basecase: ##   0:00:00  Solver: z3
SBY  5:50:35 [sby/up_down_counter_rst_max_4] engine_0.basecase: ##   0:00:00  Checking assertions in step 0..
SBY  5:50:35 [sby/up_down_counter_rst_max_4] engine_0.basecase: ##   0:00:00  Checking assumptions in step 1..
SBY  5:50:35 [sby/up_down_counter_rst_max_4] engine_0.basecase: ##   0:00:00  Checking assertions in step 1..
SBY  5:50:35 [sby/up_down_counter_rst_max_4] engine_0.basecase: ##   0:00:00  Checking assumptions in step 2..
SBY  5:50:35 [sby/up_down_counter_rst_max_4] engine_0.basecase: ##   0:00:00  Checking assertions in step 2..
SBY  5:50:35 [sby/up_down_counter_rst_max_4] engine_0.basecase: ##   0:00:00  Status: passed
SBY  5:50:35 [sby/up_down_counter_rst_max_4] engine_0.basecase: finished (returncode=0)
SBY  5:50:35 [sby/up_down_counter_rst_max_4] engine_0.basecase: Status returned by engine for basecase: pass
SBY  5:50:35 [sby/up_down_counter_rst_max_15] engine_0.basecase: ##   0:00:00  Checking assumptions in step 0..
```

```
SBY  5:50:35 [sby/up_down_counter_rst_max_4] engine_0.basecase: Status returned by engine for basecase: pass
SBY  5:50:35 [sby/up_down_counter_rst_max_15] engine_0.basecase: ##   0:00:00  Checking assumptions in step 0..
SBY  5:50:35 [sby/up_down_counter_rst_max_15] engine_0.basecase: ##   0:00:00  Checking assertions in step 0..
SBY  5:50:35 [sby/up_down_counter_rst_max_15] engine_0.basecase: ##   0:00:00  Checking assumptions in step 1..
SBY  5:50:35 [sby/up_down_counter_rst_max_15] engine_0.basecase: ##   0:00:00  Checking assertions in step 1..
SBY  5:50:35 [sby/up_down_counter_rst_max_15] engine_0.basecase: ##   0:00:00  Checking assumptions in step 2..
SBY  5:50:35 [sby/up_down_counter_rst_max_15] engine_0.basecase: ##   0:00:00  Checking assertions in step 2..
SBY  5:50:35 [sby/up_down_counter_rst_max_15] engine_0.basecase: ##   0:00:00  Status: passed
SBY  5:50:35 [sby/up_down_counter_rst_max_15] engine_0.basecase: finished (returncode=0)
SBY  5:50:35 [sby/up_down_counter_rst_max_15] engine_0.basecase: Status returned by engine for basecase: pass
SBY  5:50:35 [sby/up_down_counter_rst_max_60] engine_0.induction: ##   0:00:00  Solver: z3
SBY  5:50:35 [sby/up_down_counter_rst_max_60] engine_0.induction: ##   0:00:00  Trying induction in step 3..
SBY  5:50:35 [sby/up_down_counter_rst_max_60] engine_0.induction: ##   0:00:00  Trying induction in step 2..
SBY  5:50:35 [sby/up_down_counter_rst_max_60] engine_0.induction: ##   0:00:00  Trying induction in step 1..
SBY  5:50:35 [sby/up_down_counter_rst_max_4] engine_0.induction: ##   0:00:00  Trying induction in step 3..
SBY  5:50:35 [sby/up_down_counter_rst_max_4] engine_0.induction: ##   0:00:00  Trying induction in step 2..
SBY  5:50:35 [sby/up_down_counter_rst_max_4] engine_0.induction: ##   0:00:00  Trying induction in step 1..
SBY  5:50:35 [sby/up_down_counter_rst_max_4] engine_0.induction: ##   0:00:00  Temporal induction successful.
SBY  5:50:35 [sby/up_down_counter_rst_max_4] engine_0.induction: ##   0:00:00  Status: passed
SBY  5:50:35 [sby/up_down_counter_rst_max_4] engine_0.induction: finished (returncode=0)
SBY  5:50:35 [sby/up_down_counter_rst_max_4] engine_0.induction: Status returned by engine for induction: pass
SBY  5:50:35 [sby/up_down_counter_rst_max_60] engine_0.induction: ##   0:00:00  Temporal induction successful.
SBY  5:50:35 [sby/up_down_counter_rst_max_60] engine_0.induction: ##   0:00:00  Status: passed
SBY  5:50:35 [sby/up_down_counter_rst_max_60] engine_0.induction: finished (returncode=0)
SBY  5:50:35 [sby/up_down_counter_rst_max_60] engine_0.induction: Status returned by engine for induction: pass
SBY  5:50:35 [sby/up_down_counter_rst_max_4] summary: Elapsed clock time [H:MM:SS (secs)]: 0:00:00 (0)
SBY  5:50:35 [sby/up_down_counter_rst_max_4] summary: Elapsed process time [H:MM:SS (secs)]: 0:00:00 (0)
SBY  5:50:35 [sby/up_down_counter_rst_max_4] summary: engine_0 (smtbmc z3) returned pass for basecase
SBY  5:50:35 [sby/up_down_counter_rst_max_4] summary: engine_0 (smtbmc z3) returned pass for induction
SBY  5:50:35 [sby/up_down_counter_rst_max_4] summary: engine_0 did not produce any traces
SBY  5:50:35 [sby/up_down_counter_rst_max_4] summary: successful proof by k-induction.
SBY  5:50:35 [sby/up_down_counter_rst_max_4] DONE (PASS, rc=0)
SBY  5:50:35 [sby/up_down_counter_rst_max_15] engine_0.induction: ##   0:00:00  Solver: z3
SBY  5:50:35 [sby/up_down_counter_rst_max_15] engine_0.induction: ##   0:00:00  Trying induction in step 3..
SBY  5:50:35 [sby/up_down_counter_rst_max_15] engine_0.induction: ##   0:00:00  Trying induction in step 2..
SBY  5:50:35 [sby/up_down_counter_rst_max_15] engine_0.induction: ##   0:00:00  Trying induction in step 1..
SBY  5:50:35 [sby/up_down_counter_rst_max_15] engine_0.induction: ##   0:00:00  Temporal induction successful.
SBY  5:50:35 [sby/up_down_counter_rst_max_15] engine_0.induction: ##   0:00:00  Status: passed
SBY  5:50:35 [sby/up_down_counter_rst_max_15] engine_0.induction: finished (returncode=0)
SBY  5:50:35 [sby/up_down_counter_rst_max_15] engine_0.induction: Status returned by engine for induction: pass
SBY  5:50:35 [sby/up_down_counter_rst_max_15] summary: Elapsed clock time [H:MM:SS (secs)]: 0:00:00 (0)
SBY  5:50:35 [sby/up_down_counter_rst_max_15] summary: Elapsed process time [H:MM:SS (secs)]: 0:00:00 (0)
SBY  5:50:35 [sby/up_down_counter_rst_max_15] summary: engine_0 (smtbmc z3) returned pass for basecase
SBY  5:50:35 [sby/up_down_counter_rst_max_15] summary: engine_0 (smtbmc z3) returned pass for induction
SBY  5:50:35 [sby/up_down_counter_rst_max_15] summary: engine_0 did not produce any traces
SBY  5:50:35 [sby/up_down_counter_rst_max_15] summary: successful proof by k-induction.
SBY  5:50:35 [sby/up_down_counter_rst_max_15] DONE (PASS, rc=0)
SBY  5:50:35 [sby/up_down_counter_rst_max_60] engine_0.basecase: ##   0:00:00  Solver: z3
SBY  5:50:35 [sby/up_down_counter_rst_max_60] engine_0.basecase: ##   0:00:00  Checking assumptions in step 0..
SBY  5:50:35 [sby/up_down_counter_rst_max_60] engine_0.basecase: ##   0:00:00  Checking assertions in step 0..
SBY  5:50:35 [sby/up_down_counter_rst_max_60] engine_0.basecase: ##   0:00:00  Checking assumptions in step 1..
SBY  5:50:35 [sby/up_down_counter_rst_max_60] engine_0.basecase: ##   0:00:00  Checking assertions in step 1..
SBY  5:50:35 [sby/up_down_counter_rst_max_60] engine_0.basecase: ##   0:00:00  Checking assumptions in step 2..
SBY  5:50:35 [sby/up_down_counter_rst_max_60] engine_0.basecase: ##   0:00:00  Checking assertions in step 2..
SBY  5:50:35 [sby/up_down_counter_rst_max_60] engine_0.basecase: ##   0:00:00  Status: passed
SBY  5:50:35 [sby/up_down_counter_rst_max_60] engine_0.basecase: finished (returncode=0)
```

```
SBY  5:50:35 [sby/up_down_counter_rst_max_60] engine_0.basecase: finished (returncode=0)
SBY  5:50:35 [sby/up_down_counter_rst_max_60] engine_0.basecase: Status returned by engine for basecase: pass
SBY  5:50:35 [sby/up_down_counter_rst_max_60] summary: Elapsed clock time [H:MM:SS (secs)]: 0:00:00 (0)
SBY  5:50:35 [sby/up_down_counter_rst_max_60] summary: Elapsed process time [H:MM:SS (secs)]: 0:00:00 (0)
SBY  5:50:35 [sby/up_down_counter_rst_max_60] summary: engine_0 (smtbmc z3) returned pass for basecase
SBY  5:50:35 [sby/up_down_counter_rst_max_60] summary: engine_0 (smtbmc z3) returned pass for induction
SBY  5:50:35 [sby/up_down_counter_rst_max_60] summary: engine_0 did not produce any traces
SBY  5:50:35 [sby/up_down_counter_rst_max_60] summary: successful proof by k-induction.
SBY  5:50:35 [sby/up_down_counter_rst_max_60] DONE (PASS, rc=0)
root@0b9c66e66b04:/workspace#
```


root@0b9c66e66b04:/workspace# pytest -m formal -v
test session starts
platform linux -- Python 3.12.3, pytest-9.0.3, pluggy-1.7.0 -- /opt/venv/bin/python3
cachedir: .pytest_cache
metadata: {'Python': '3.12.3', 'Platform': 'Linux-7.0.12-linuxkit-x86_64-with-glibc2.39', 'Packages': {'pytest': '9.0.3', 'pluggy': '1.7.0'}, 'Plu
gins': {'metadata': '3.1.1', 'html': '4.2.0'}}
rootdir: /workspace
configfile: .pytest.toml
testpaths: tests
plugins: metadata-3.1.1, html-4.2.0
collected 114 items / 106 deselected / 8 selected
tests/test_formal.py::test_formal_sby[cascade_counter] SKIPPED (cascade_counter module not implemented yet) [12%]
tests/test_formal.py::test_formal_sby[editable_countdown] SKIPPED (editable_countdown module not implemented yet) [25%]
tests/test_formal.py::test_formal_sby[snapshot_mux] SKIPPED (snapshot_mux module not implemented yet) [37%]
tests/test_formal.py::test_formal_sby[stopwatch_control] SKIPPED (stopwatch_control module not implemented yet) [50%]
tests/test_formal.py::test_formal_sby[stopwatch_counter] SKIPPED (stopwatch_counter module not implemented yet) [62%]
tests/test_formal.py::test_formal_sby[up_down_counter_rst] PASSED [75%]
tests/test_formal.py::test_formal_sby[user_top_brightness_wrapper] SKIPPED (user_top_brightness_wrapper module not implemented yet) [87%]
tests/test_formal.py::test_formal_sby[user_top_timer_v1] SKIPPED (user_top_timer_v1 module not implemented yet) [100%]
1 passed, 7 skipped, 106 deselected in 0.31s
root@0b9c66e66b04:/workspace#

3.2. Assertion File

The assertion file wraps the design under verification in a module that instantiates it and adds the formal properties to be checked. Because SVA concurrent assertions (which use the `property` and `sequence` keywords to express temporal properties) require the commercial Verific front-end and are therefore not available in open-source Yosys flows, we use immediate assertions instead: `assert` statements placed directly inside procedural blocks.

When a design passes formal verification, it is worth reflecting on what has actually been proved. The proof establishes not that this module *is* an up-down counter with reset, but that it satisfies the properties captured by the assertions above. Whether those properties fully capture the concept of an up-down counter is a question about language, not logic; a lawyer might ask whether any reasonable person would consider the specification complete. Engineers can afford to be more pragmatic: provided nothing more is asked of this module than what the assertions guarantee, it is fit for purpose.

Report 2

Do the assertions above fully capture the behaviour of an up-down counter with reset? If not, identify any behaviours left unspecified, or any properties a reasonable person might expect that the assertions do not guarantee.

Report 3

The configuration file verifies the counter for three specific values of `MAX`. Since this does not cover all possible values, how would you formally prove correctness when this module is used in a larger design?

reset has two states: HIGH and LOW
are there any input sequence involving `reset` that makes any asserts fail the current cycle?

enumerates all possible combination of signal states and runs the transitions

use BMC and k-induction

3.3. Assertions, Valid States and Verification Checks

The assertions are checked once per clock cycle. The formal verification tools assume ideal logic: signal values update just after a rising edge and remain constant until the next.

A state at cycle n is *valid* if no assertions fail at that cycle.

The two checks use this definition differently, as we now describe.

The formal verification tools run two checks: a bounded model check (BMC), which verifies that no assertion fails in the first k cycles, and a k-induction check, which verifies that any sequence of k consecutive valid states must be followed by another valid state. When both pass at the same depth k, together they constitute a complete proof of correctness for all time: the BMC establishes that the first k steps are assertion-free, and the induction check extends this guarantee indefinitely.

We use assertions in three block types: `initial`, `always @(*)`, and `always @(posedge clk)`. Each produces a counterexample when it fails; interpreting these correctly requires understanding how the two checks treat each block type. Since signals are constant within a cycle, it might seem that `always @(*)` and `always @(posedge clk)` are interchangeable. They are not.

An assertion in an `initial` block applies only to cycle 0. The induction checker ignores it, since induction requires assertions that can apply starting from any point in time, not just the initial state.

An assertion in an `always @(*)` block applies to all cycles, including cycle 0. Both checkers consider a state at cycle 0 to be invalid if it fails any such assertion, just as one would expect.

An assertion in an `always @(posedge clk)` block is attributed to the cycle being entered: the first time it triggers, it is attributed to cycle 1. Two things are natural: it is natural for the checker to attribute the assertion to the new cycle, and it is equally natural that the assertion checks the state of the previous cycle, as explained below. This mismatch does not compromise the validity of formal verification, but readers should keep it in mind when interpreting counterexample output.

Rising edges demarcate cycles: each edge marks the end of one cycle and the beginning of the next. The period before an edge is the *previous cycle*; the period after it is the *current cycle*. The first time `always @(posedge clk)` triggers, the circuit enters cycle 1. The checker adopts this labelling, which is entirely natural.

Because flip-flop outputs are assigned via non-blocking assignment, they take effect only after all procedural blocks have finished. Inside an `always @(posedge clk)` block, signal values are therefore those from just before the rising edge, that is, from the previous cycle. In particular, at cycle 1 the assertions check the state at cycle 0.

To make this concrete, suppose the induction checker initialises the state at cycle 0 to `start=1` and `count=3`. No assertions are violated because no rising edge has yet occurred. When the first rising edge arrives and line 45 executes, the prover records a violation at cycle 1, but the condition it checks concerns cycle 0. Someone reading "invalid at step 1" would naturally examine cycle 1, while the actual problem lies in cycle 0.

The k-induction check for our counter fails when $k = 1$, an artefact of the mismatched labelling. From the counter's logic alone, if the state at cycle k is valid then the state at cycle $k + 1$ will also be valid, so one would expect the counter to pass 1-induction. The subtlety is that 1-induction is genuine 1-induction for `always @(*)` assertions, but behaves like 0-induction for `always @(posedge clk)` assertions, since those check the previous cycle's state. The counter therefore requires 2-induction to pass.

To see why, consider the 1-induction counterexample. The checker places `start=1` and `count=3` at cycle 0. No `always @(*)` assertions fail, so cycle 0 is deemed valid. At the first rising edge, labelled cycle 1, line 45 executes and finds `count` non-zero at cycle 0, recording

a violation at cycle 1. A valid cycle 0 has thus led to a violation at cycle 1, so 1-induction fails, even though the counter is correct.

This artefact does not compromise the soundness of formal verification. The looser constraint on cycle 0 gives the induction checker more freedom to construct an adversarial initial state, making induction *harder* to pass, not easier. Any design that does pass is genuinely correct; the artefact merely forces a higher depth. For our counter, $k = 2$ suffices: the 2-induction hypothesis covers two consecutive valid states, which is enough to exclude the spurious counterexample above.

3.4. Configuration File

The configuration file `sby/up_down_counter_rst.sby` is shown below, followed by a discussion of each part keyed to the line numbers.

Lines 1–3 set the proof depth to 3, which governs both checkers. The base-case checker verifies that no assertion is violated in cycles 0, 1, or 2. The induction checker searches for a different kind of counterexample: does there exist a run of three consecutive cycles with all assertions satisfied, immediately followed by a cycle that violates one? If no such run exists, the property holds for all time.

3.5. Expected Output

Because `sby` cannot verify over all possible parameter values, we select a representative sample of `MAX` values and define each as a separate task in the configuration file `up_down_counter_rst.sby`: `max_4`, `max_15`, and `max_60`. When run without a task name, the command above launches all tasks in parallel, producing interleaved output; the third column of each line identifies which task produced it.

The configuration file specifies a depth of 3, which governs both checks. The base-case checker verifies that no assertion fails at clock cycles 0, 1, or 2, starting from the known initial state. The induction checker runs independently, without constraining the starting state to be the initial state: it asks whether there exist any three consecutive cycles in which all assertions hold, but the successor cycle violates one. If no such sequence exists, the inductive step holds, and together with the base case, k-induction guarantees the assertions hold for all time.

The induction checker's output was interleaved with the previous block but finishes here. Its goal is to prove that no assertion is ever violated: for all n, no assertion fails at cycle n. Unlike the base-case checker, it imposes no constraint on the starting state, so the proof must cover all possible mid-execution states, not just those reachable immediately from startup.

In summary, each step widens the window of required valid states before the potential violation, restricting the set of starting states the checker must consider. Once that set is narrow enough that no violating run survives, the inductive step is proved. Together with the base-case result, the k-induction proof is complete.

3.6. Examining Counterexamples

To see what a failing verification looks like, consider an up-down counter in which the count-down wrap is omitted: rather than wrapping to `MAX` when `count` reaches zero, the code subtracts one unconditionally and lets the bit-vector underflow. Excerpts of the `sby` output for the `max_4` task are shown below in three parts, with prefixes and irrelevant lines removed.

The failed assertion is the range check on line 38 (`count <= MAX`). The trace is the counterexample `sby` found: the inputs `enable = 1`, `rst = 0`, and `up = 0` are applied while `count = 0`, and since the wrap is missing, `count` underflows to $2^{\texttt{WIDTH}} - 1$, which exceeds `MAX`.

The induction checker fails for the same reason. It searches for an arbitrary starting state and a sequence of inputs such that no assertion is violated in the first three cycles, but one is violated on the fourth; `count` can begin at any value the solver finds convenient.

Its counterexample is saved to `trace_induct.vcd`: opening it in VaporView shows `count` taking the values 3, 4, 0, then 7. Starting from `count = 3`, three up-count steps give 3, 4, 0, all within range (the wrap is correctly implemented); a final down-count step then causes `count` to underflow to 7, violating the range assertion.

The final summary confirms that both the base-case and induction checks failed, identifies the paths to the two counterexample trace files, and reports the specific assertion that was violated in each case, including its source location (`up_down_counter_rst_assert.sv`, line 38). The overall result is `FAIL` with return code 2, and `sby` lists the tasks that failed. The return code is useful when `sby` is run as part of an automated flow: a non-zero exit code signals failure to the calling script or build system.

Finding Counterexample Waveforms. Whenever `sby` reports a failure, you will need to open the counterexample trace files in VaporView. The paths to these files are printed in `sby`'s output, on lines labelled `counterexample trace`. For this run they are `sby/up_down_counter_rst_max_4/engine_0/trace.vcd` (base case) and `sby/up_down_counter_rst_max_4/engine_0/trace_induct.vcd` (induction). In future runs the filenames will differ, so read them directly from the output.

Valid Versus Reachable. The discussion below of k-induction's limitations hinges on the distinction between these two notions. The following example makes it concrete.

```
1  always_comb
2    if (count < 10) next_count = (count == 9) ? 0 : count + 1;
3    else next_count = (count == 19) ? 10 : count + 1;
```

This defines two independent counters: one cycling through 0–9 and the other through 10–19. No sequence of inputs can move the system from one range to the other. If `count` is initialised to 0, only the states 0–9 are reachable; the states 10–19 are not. Formally, a state is *reachable* if the design can arrive at it from the initial state defined by Verilog `initial` statements. A state is *valid* if it violates no assertion (Section 3.3). A *valid sequence* is a sequence of valid states in which each consecutive pair is connected by a transition that the design permits under some input. Not every valid state need be reachable.

Limitations of k-induction. Because the induction checker starts from an arbitrary state, its counterexamples are not guaranteed to be reachable. A *spurious* counterexample is one whose starting state is valid but unreachable from the true initial state of the design. When a spurious counterexample is present, the induction step fails even if the design is correct, and the proof cannot succeed until it is ruled out.

The simplest approach is to try increasing the depth: a longer lead-in of valid states is harder to construct, so some spurious counterexamples are ruled out as the depth grows.

However, this does not always work, particularly for designs with an `enable` input. A spurious counterexample found at one depth can be adapted to any greater depth by prepending additional cycles with `enable` held low (since nothing changes while `enable` is low, each such cycle remains valid). This exact problem arises in our counter if the following assertion is removed:

```
1 always @(*) assert (count <= MAX);
```

Without it, the induction checker cannot succeed at any depth. It can initialise `count` to a value greater than `MAX`, then hold `enable` low indefinitely without violating any assertion. Setting `enable` and `up` high then causes `count` to wrap to zero, but because it wraps from a value other than `MAX`, the transition violates the count-up assertion.

Task Temporarily comment out the `assert (count <= MAX)` line (shortcut: Ctrl+/), run `sby`, and observe that the induction step now fails. The paths to the counterexample trace files are printed in `sby`'s output; open them in VaporView. **Restore the assertion file before continuing.**

Report 4

Include the counterexample waveform. The general reason for this failure was explained above; use the waveform to identify the specific counterexample the checker found and explain why it is one. Then explain why restoring the assertion allows the induction step to succeed.

```
11  SBY 11:28:03 [sby/up_down_counter_rst_max_4] engine_0.basecase: starting process "cd sby/up_down_counter_rst_max_4; yosys-smtbmc -s z3 --presat --noprogress -t 3  --append
12  SBY 11:28:03 [sby/up_down_counter_rst_max_4] engine_0.induction: starting process "cd sby/up_down_counter_rst_max_4; yosys-smtbmc -s z3 --presat -i --noprogress -t 3  --ap
13  SBY 11:28:03 [sby/up_down_counter_rst_max_4] engine_0.basecase: ##   0:00:00  Solver: z3
14  SBY 11:28:03 [sby/up_down_counter_rst_max_4] engine_0.basecase: ##   0:00:00  Checking assumptions in step 0..
15  SBY 11:28:03 [sby/up_down_counter_rst_max_4] engine_0.basecase: ##   0:00:00  Checking assertions in step 0..
16  SBY 11:28:03 [sby/up_down_counter_rst_max_4] engine_0.basecase: ##   0:00:00  Checking assumptions in step 1..
17  SBY 11:28:03 [sby/up_down_counter_rst_max_4] engine_0.basecase: ##   0:00:00  Checking assertions in step 1..
18  SBY 11:28:03 [sby/up_down_counter_rst_max_4] engine_0.basecase: ##   0:00:00  Checking assumptions in step 2..
19  SBY 11:28:03 [sby/up_down_counter_rst_max_4] engine_0.basecase: ##   0:00:00  Checking assertions in step 2..
20  SBY 11:28:03 [sby/up_down_counter_rst_max_4] engine_0.basecase: ##   0:00:00  Status: passed
21  SBY 11:28:03 [sby/up_down_counter_rst_max_4] engine_0.basecase: finished (returncode=0)
22  SBY 11:28:03 [sby/up_down_counter_rst_max_4] engine_0.basecase: Status returned by engine for basecase: pass
23  SBY 11:28:03 [sby/up_down_counter_rst_max_4] engine_0.induction: ##   0:00:00  Solver: z3
24  SBY 11:28:03 [sby/up_down_counter_rst_max_4] engine_0.induction: ##   0:00:00  Trying induction in step 3..
25  SBY 11:28:03 [sby/up_down_counter_rst_max_4] engine_0.induction: ##   0:00:00  Trying induction in step 2..
26  SBY 11:28:03 [sby/up_down_counter_rst_max_4] engine_0.induction: ##   0:00:00  Trying induction in step 1..
27  SBY 11:28:03 [sby/up_down_counter_rst_max_4] engine_0.induction: ##   0:00:00  Trying induction in step 0..
28  SBY 11:28:03 [sby/up_down_counter_rst_max_4] engine_0.induction: ##   0:00:00  Temporal induction failed!
29  SBY 11:28:03 [sby/up_down_counter_rst_max_4] engine_0.induction: ##   0:00:00  Assert failed in up_down_counter_rst_assert: a_count_up
30  SBY 11:28:03 [sby/up_down_counter_rst_max_4] engine_0.induction: ##   0:00:00  Writing trace to VCD file: engine_0/trace_induct.vcd
31  SBY 11:28:03 [sby/up_down_counter_rst_max_4] engine_0.induction: ##   0:00:00  Writing trace to Verilog testbench: engine_0/trace_induct_tb.v
32  SBY 11:28:03 [sby/up_down_counter_rst_max_4] engine_0.induction: ##   0:00:00  Writing trace to constraints file: engine_0/trace_induct.smtc
33  SBY 11:28:03 [sby/up_down_counter_rst_max_4] engine_0.induction: ##   0:00:00  Writing trace to Yosys witness file: engine_0/trace_induct.yw
34  SBY 11:28:03 [sby/up_down_counter_rst_max_4] engine_0.induction: ##   0:00:00  Status: failed
35  SBY 11:28:03 [sby/up_down_counter_rst_max_4] engine_0.induction: finished (returncode=1)
36  SBY 11:28:03 [sby/up_down_counter_rst_max_4] engine_0.induction: Status returned by engine for induction: FAIL
37  SBY 11:28:03 [sby/up_down_counter_rst_max_4] summary: Elapsed clock time [H:MM:SS (secs)]: 0:00:00 (0)
38  SBY 11:28:03 [sby/up_down_counter_rst_max_4] summary: Elapsed process time [H:MM:SS (secs)]: 0:00:00 (0)
39  SBY 11:28:03 [sby/up_down_counter_rst_max_4] summary: engine_0 (smtbmc z3) returned pass for basecase
40  SBY 11:28:03 [sby/up_down_counter_rst_max_4] summary: engine_0 (smtbmc z3) returned FAIL for induction
41  SBY 11:28:03 [sby/up_down_counter_rst_max_4] summary: counterexample trace [induction]: sby/up_down_counter_rst_max_4/engine_0/trace_induct.vcd
42  SBY 11:28:03 [sby/up_down_counter_rst_max_4] summary:   failed assertion up_down_counter_rst_assert.a_count_up at up_down_counter_rst_assert.sv:63.7-63.65
43  SBY 11:28:03 [sby/up_down_counter_rst_max_4] DONE (UNKNOWN, rc=4)
44
```

```
sby > up_down_counter_rst_max_4 > UNKNOWN
1  Elapsed clock time [H:MM:SS (secs)]: 0:00:00 (0)
2  Elapsed process time [H:MM:SS (secs)]: 0:00:00 (0)
3  engine_0 (smtbmc z3) returned pass for basecase
4  engine_0 (smtbmc z3) returned FAIL for induction
5  counterexample trace [induction]: engine_0/trace_induct.vcd
6  counterexample trace [induction]: engine_0/trace_induct.yw
7    failed assertion up_down_counter_rst_assert.a_count_up at up_down_counter_rst_assert.sv:63.7-63.65
8
```

```
SBY 11:28:03 [sby/up_down_counter_rst_max_60] summary: engine_0 (smtbmc z3) returned pass for basecase
SBY 11:28:03 [sby/up_down_counter_rst_max_60] summary: engine_0 (smtbmc z3) returned FAIL for induction
SBY 11:28:03 [sby/up_down_counter_rst_max_60] summary: counterexample trace [induction]: sby/up_down_counter_rst_max_60/engine_0/trace_induct.vcd
SBY 11:28:03 [sby/up_down_counter_rst_max_60] summary:   failed assertion up_down_counter_rst_assert.a_count_up at up_down_counter_rst_assert.sv:63.7-63.65
SBY 11:28:03 [sby/up_down_counter_rst_max_60] DONE (UNKNOWN, rc=4)
SBY 11:28:03 The following tasks failed: ['max_4', 'max_60']
root@0b9c66e66b04:/workspace#
```

sby > up_down_counter_rst_max_4 > engine_0 > logfile_induction.txt

```
1   ##   0:00:00  Solver: z3
2   ##   0:00:00  Trying induction in step 3..
3   ##   0:00:00  Trying induction in step 2..
4   ##   0:00:00  Trying induction in step 1..
5   ##   0:00:00  Trying induction in step 0..
6   ##   0:00:00  Temporal induction failed!
7   ##   0:00:00  Assert failed in up_down_counter_rst_assert: a_count_up
8   ##   0:00:00  Writing trace to VCD file: engine_0/trace_induct.vcd
9   ##   0:00:00  Writing trace to Verilog testbench: engine_0/trace_induct_tb.v
10  ##   0:00:00  Writing trace to constraints file: engine_0/trace_induct.smtc
11  ##   0:00:00  Writing trace to Yosys witness file: engine_0/trace_induct.yw
12  ##   0:00:00  Status: failed
13
```

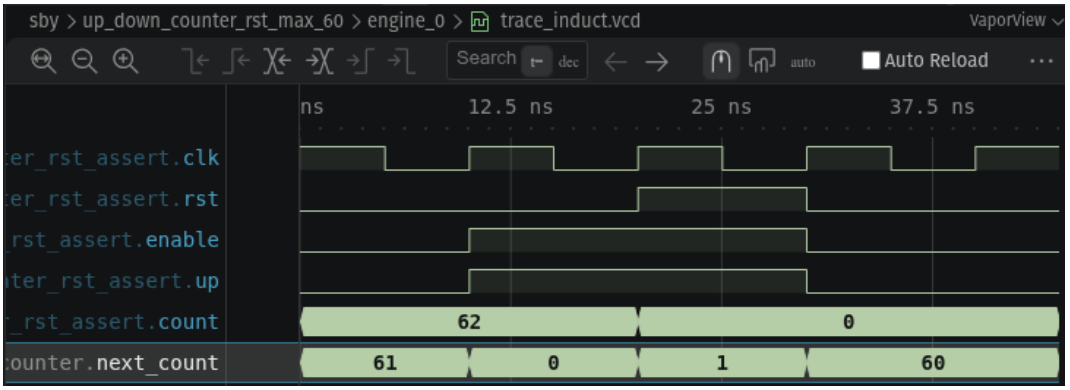
rtl > up_down_counter_rst.sv > ...

```
51  always_comb begin
52    if (up) begin  // up=1 -> Count up
53      if (count < Max) next_count = count + WIDTH'(1);
54      else next_count = WIDTH'(0);  // Wrap to 0 when Max
55    end else begin  // else up=0 -> Count down
56      if (count > WIDTH'(0)) next_count = count - WIDTH'(1);
57      else next_count = Max;  // Wrap to Max when 0
58    end
59  end
```

sby > up_down_counter_rst_assert.sv > ...

```
58  // Check count up
59  logic wrap_up;
60  always @(posedge clk) begin
61    wrap_up = ($past(count == MAX) && count == 0);
62    if (!start && $past(up && enable && !rst))
63      a_count_up : assert (count == $past(count) + 1 || wrap_up);
64  end
```





MAX_4
- [2:0] count = 5 is a possible state (3'b101)
- assert `a_count_up` expects count to be next reachable 6 at k-2 (step 2/cycle 2) but is 0 due to next_count = 0 at k-1 (step 1/cycle 1) by the next stage logic in the DUT module. Since 0 =/= 5+1, assert fails. Although the next state 0 is reachable, the transition is invalid and thus count 5 to count 0 is an invalid sequence.
- If assert (count<=MAX) exists, the starting states of `count` will be restricted to 0-4

Feasibility of Formal Verification. In principle, formal verification of finite-state designs such as digital circuits is decidable: for a correct design, there exists an inductive invariant that proves the property. In practice, however, two barriers limit its applicability. First, the search space grows exponentially with the number of state bits, so large designs may be computationally intractable. Second, finding the necessary assertion strengthening for k-induction cannot in general be automated: the auxiliary assertions needed to rule out spurious counterexamples may be non-obvious and require human insight. At the scale of this project, a complete proof is typically achievable. For industrial-scale designs, formal verification remains an active research area, and engineers sometimes accept bounded model checking as a practical substitute.

4. Stopwatch

In Assignment 1 we developed a settable watch; by the end of this assignment it will be extended with a stopwatch, a timer, and display brightness control. Because these features are loosely coupled, we begin by setting the settable watch aside and focusing purely on the stopwatch.

The board wrapper from Assignment 1 is reused here, so we need only concern ourselves with the interface defined by the `user_top` module created there.

4.1. Specification

The stopwatch displays minutes, seconds, and centiseconds on the seven-segment displays. The two buttons operate as follows.

Button	Function
`button[0]`	Start / Stop
`button[1]`	Lap (while running) / Reset (while stopped)

Start/Stop (`button[0]`) Pressing `button[0]` toggles the stopwatch between running and stopped. It does not affect the display state (live or frozen). The first centisecond increment must occur 10 ms after the button press, not sooner.

Lap and Reset (`button[1]`) The behaviour of `button[1]` depends on the current state:

- While running with a live display, pressing `button[1]` freezes the display at the current time; the stopwatch continues running in the background.
- While running with a frozen display, pressing `button[1]` unfreezes the display.
- While stopped with a frozen display, pressing `button[1]` unfreezes the display; the time is not reset.
- While stopped with a live display, pressing `button[1]` resets the time to zero; the stopwatch remains stopped and the display remains live.

Simultaneous Presses If both buttons are pressed on the same clock edge, both are ignored.

4.2. Design Approach

While this sounds straightforward, particularly given that modules from Assignment 1 can be reused, implementing the stopwatch as a single monolithic module is harder than it appears. A well-decomposed design keeps each module focused on at most one or two non-trivial decisions.

Report 5

Before reading further, sketch how you would approach the design.

Report 6

Using a feature layering approach, how would you decompose the stopwatch into layers? Describe the responsibility of each layer.

The resulting design has a natural structure. The *data path* (the chain of modules through which the time value flows toward the display) runs from the stopwatch counter through the snapshot multiplexer to the display driver in the board wrapper. Alongside it sits a small set of *control signals*: reset and enable for the counter, and a freeze signal for the multiplexer. A separate module will drive these via a simple FSM that responds to button presses.

Report 7

Draw a block diagram of the design described above, clearly showing the data path and control signals. Compare it with your initial sketch from Section 4.2 and comment on any differences.

Because `stopwatch_counter` has a non-trivial timing constraint, we keep the implementation concise by factoring out a cascade counter as a separate module. It will be driven by the `restartable_rate_generator` from Assignment 1.

4.3. Cascade Counter

All outputs are initialised to 0.

When `rst` is high, all outputs are set to 0 on the next rising clock edge, regardless of `enable`.

When `rst` is low and `enable` is low, all outputs are unchanged.

When `rst` is low and `enable` is high, `count0` increments on each rising clock edge, wrapping from `N0-1` to 0. When `count0` wraps, `count1` increments, wrapping from `N1-1` to 0. When `count1` wraps, `count2` increments, wrapping from `N2-1` to 0.

Dependencies Only the following module is to be directly instantiated.

- `mod_n_counter`

rtl > cascade_counter.sv > ...

```sv
// Cascade Counter
// Consists of three cascading Modulo N Counters that syncs three sets of
// counters together, where the rollover of initial counter performs the tick
// for the next counter.
//
// Parameters:
//    N2:    - Max 'length' value for Counter N2
//    N1:    - Max 'length' value for Counter N1
//    N0:    - Max 'length' value for Counter N0
//    W2:    - Output port bit width of `N2-1` value
//    W1:    - Output port bit width of `N1-1` value
//    W0:    - Output port bit width of `N0-1` value
//
// Ports:
//    clk:              - Clock input
//    rst:              - Asynchronous reset input
//    enable:           - When HIGH, module is enabled
//    [W2-1:0] count2:  - Output Signal of counter N2 (0 to N2-1)
//    [W1-1:0] count1:  - Output Signal of counter N1 (0 to N1-1)
//    [W0-1:0] count0:  - Output Signal of counter N0 (0 to N0-1)
//
// Local Variables:
//    CountN2, CountN1, CountN1: - To hold their respective parameter values
//                                 (N2-1), (N1-1), (N0-1) with their correct
//                                 bit widths
//    enable1:  - HIGH only when N0 rolls over and enable is HIGH
//    enable2:  - HIGH only when N1 rolls over and enable1 is HIGH

`timescale 1ns / 1ps

module cascade_counter #(
    parameter int N2 = 3,
    parameter int N1 = 4,
    parameter int N0 = 5,

    // Output port widths
    parameter int W2 = 2,
    parameter int W1 = 2,
    parameter int W0 = 3
) (
    input logic clk,
    input logic rst,
    input logic enable,
    output logic [W2-1:0] count2,
    output logic [W1-1:0] count1,
    output logic [W0-1:0] count0
);

  localparam logic [W2-1:0] CountN2 = (W2)'(N2) - 1;
  localparam logic [W1-1:0] CountN1 = (W1)'(N1) - 1;
```

```
    localparam logic [W1-1:0] CountN1 = (W1)'(N1) - 1;
    localparam logic [W0-1:0] CountN0 = (W0)'(N0) - 1;
    logic enable1, enable2;

    mod_n_counter #(
        .N(CountN0 + 1),
        .WIDTH(W0)
    ) u_count_N0 (
        .clk(clk),
        .rst(rst),
        .enable(enable),
        .count(count0)
    );

    mod_n_counter #(
        .N(CountN1 + 1),
        .WIDTH(W1)
    ) u_count_N1 (
        .clk(clk),
        .rst(rst),
        .enable(enable1),
        .count(count1)
    );

    mod_n_counter #(
        .N(CountN2 + 1),
        .WIDTH(W2)
    ) u_count_N2 (
        .clk(clk),
        .rst(rst),
        .enable(enable2),
        .count(count2)
    );

    always_comb begin
      if (count0 == CountN0 && enable) enable1 = 1'b1;
      else enable1 = 1'b0;
      if (count1 == CountN1 && enable1) enable2 = 1'b1;
      else enable2 = 1'b0;
    end

  endmodule
```

```
root@0b9c66e66b04:/workspace# verilator -Wall --lint-only -y rtl rtl/cascade_counter.sv
root@0b9c66e66b04:/workspace# iverilog -Wall -g2012 -Y .sv -y rtl rtl/cascade_counter.sv
root@0b9c66e66b04:/workspace#
```

```
SBY  5:30:38 [sby/cascade_counter_default] summary: engine_0 (smtbmc z3) returned pass for basecase
SBY  5:30:38 [sby/cascade_counter_default] summary: engine_0 (smtbmc z3) returned pass for induction
SBY  5:30:38 [sby/cascade_counter_default] summary: engine_0 did not produce any traces
SBY  5:30:38 [sby/cascade_counter_default] summary: successful proof by k-induction.
SBY  5:30:38 [sby/cascade_counter_default] DONE (PASS, rc=0)
SBY  5:30:38 [sby/cascade_counter_large] engine_0.induction: ##   0:00:00  Solver: z3
SBY  5:30:38 [sby/cascade_counter_large] engine_0.induction: ##   0:00:00  Trying induction in step 3..
SBY  5:30:38 [sby/cascade_counter_large] engine_0.induction: ##   0:00:00  Trying induction in step 2..
SBY  5:30:38 [sby/cascade_counter_large] engine_0.induction: ##   0:00:00  Trying induction in step 1..
SBY  5:30:38 [sby/cascade_counter_large] engine_0.induction: ##   0:00:00  Temporal induction successful.
SBY  5:30:38 [sby/cascade_counter_large] engine_0.induction: ##   0:00:00  Status: passed
SBY  5:30:38 [sby/cascade_counter_large] engine_0.induction: finished (returncode=0)
SBY  5:30:38 [sby/cascade_counter_large] engine_0.induction: Status returned by engine for induction: pass
SBY  5:30:38 [sby/cascade_counter_large] summary: Elapsed clock time [H:MM:SS (secs)]: 0:00:00 (0)
SBY  5:30:38 [sby/cascade_counter_large] summary: Elapsed process time [H:MM:SS (secs)]: 0:00:00 (0)
SBY  5:30:38 [sby/cascade_counter_large] summary: engine_0 (smtbmc z3) returned pass for basecase
SBY  5:30:38 [sby/cascade_counter_large] summary: engine_0 (smtbmc z3) returned pass for induction
SBY  5:30:38 [sby/cascade_counter_large] summary: engine_0 did not produce any traces
SBY  5:30:38 [sby/cascade_counter_large] summary: successful proof by k-induction.
SBY  5:30:38 [sby/cascade_counter_large] DONE (PASS, rc=0)
root@0b9c66e66b04:/workspace#
```

```
root@0b9c66e66b04:/workspace# pytest -m formal -v
============================= test session starts ==============================
platform linux -- Python 3.12.3, pytest-9.0.3, pluggy-1.7.0 -- /opt/venv/bin/pyth
on3
cachedir: .pytest_cache
metadata: {'Python': '3.12.3', 'Platform': 'Linux-7.0.12-linuxkit-x86_64-with-gli
bc2.39', 'Packages': {'pytest': '9.0.3', 'pluggy': '1.7.0'}, 'Plugins': {'metadat
a': '3.1.1', 'html': '4.2.0'}}
rootdir: /workspace
configfile: .pytest.toml
testpaths: tests
plugins: metadata-3.1.1, html-4.2.0
collected 116 items / 108 deselected / 8 selected

tests/test_formal.py::test_formal_sby[cascade_counter] PASSED             [ 12%]
tests/test_formal.py::test_formal_sby[editable_countdown] SKIPPED (ed...) [ 25%]
tests/test_formal.py::test_formal_sby[snapshot_mux] SKIPPED (snapshot...) [ 37%]
tests/test_formal.py::test_formal_sby[stopwatch_control] SKIPPED (sto...) [ 50%]
tests/test_formal.py::test_formal_sby[stopwatch_counter] SKIPPED (sto...) [ 62%]
tests/test_formal.py::test_formal_sby[up_down_counter_rst] PASSED         [ 75%]
tests/test_formal.py::test_formal_sby[user_top_brightness_wrapper] SKIPPED [ 87%]
tests/test_formal.py::test_formal_sby[user_top_timer_v1] SKIPPED (use...) [100%]

================== 2 passed, 6 skipped, 108 deselected in 0.71s ==================
root@0b9c66e66b04:/workspace#
```

4.4. Stopwatch Counter

All outputs are initialised to 0.

When `rst` is high, all outputs are set to 0 on the next rising clock edge, regardless of `enable`.

When `rst` is low and `enable` is low, all outputs are unchanged.
When `rst` is low and `enable` is high, `centiseconds` increments once per centisecond, wrapping from 99 to 0. When `centiseconds` wraps, `seconds` increments, wrapping from 59 to 0. When `seconds` wraps, `minutes` increments, wrapping from 99 to 0. The first increment must occur one centisecond after `enable` goes high, not sooner.

rtl > stopwatch_counter.sv > ...

```sv
// Stopwatch Counter
// A module that counts (99) minutes, (59) seconds and (99) centiseconds using
// the Restartable Rate Generator that ticks per 1/10th of CYCLES_PER_SECOND
// and the Cascade Counter that drives the Output Signals.
//
// Parameters:
//    CYCLES_PER_SECOND: - Number of cycles per second
//
// Ports:
//    clk:                - Clock input
//    rst:                - Asynchronous reset input
//    enable:             - When HIGH, module is enabled
//    [6:0] minutes:      - Output Signal for 7'd99 minutes
//    [5:0] seconds:      - Output Signal for 6'd59 seconds
//    [6:0] centiseconds: - Output Signal for 7'd99 centiseconds
//
// Local Variables:
//    gen_run:  - HIGH *only* when `rst`:LOW and `enable`:HIGH
//    tick:     - HIGH when u_centisecond_counter completes a cycle
//    tick_centisecond: - HIGH only when `tick`:HIGH and `enable`:HIGH

`timescale 1ns / 1ps

module stopwatch_counter #(
    parameter int CYCLES_PER_SECOND = 50_000_000
) (
    input logic clk,
    input logic rst,  // Takes priority over enable
    input logic enable,
    output logic [6:0] minutes,
    output logic [5:0] seconds,
    output logic [6:0] centiseconds  // hundredths of a second
);

  logic gen_run;
  logic tick;
  logic tick_centisecond;

  restartable_rate_generator #(
      .CYCLE_COUNT(CYCLES_PER_SECOND / 100)
  ) u_centisecond_counter (
      .clk (clk),
      .run (gen_run),
      .tick(tick)
  );

  cascade_counter #(
      .N0(100),
      .N1(60),
      .N2(100),
      .W0(7),
      .W1(6),
      .W2(7)
  ) u_stopwatch_counter (
      .clk(clk),
      .rst(rst),
      .enable(tick_centisecond),
      .count0(centiseconds),
      .count1(seconds),
      .count2(minutes)
  );

  always_comb begin
    if (rst || !enable) begin
      gen_run = 1'b0;
      tick_centisecond = 1'b0;
    end else begin
      gen_run = 1'b1;
      tick_centisecond = tick;
    end
  end

endmodule

```

```
root@0b9c66e66b04:/workspace# verilator -Wall --lint-only -y rtl rtl/stopwatch_counter.sv
root@0b9c66e66b04:/workspace# iverilog -Wall -g2012 -Y .sv -y rtl rtl/stopwatch_counter.sv
root@0b9c66e66b04:/workspace#
```

```
SBY  8:49:38 [sby/stopwatch_counter_div_7] summary: Elapsed process time [H:MM:SS (secs)]: 0:00:01 (1)
SBY  8:49:38 [sby/stopwatch_counter_div_7] summary: engine_0 (smtbmc z3) returned pass for basecase
SBY  8:49:38 [sby/stopwatch_counter_div_7] summary: engine_0 (smtbmc z3) returned pass for induction
SBY  8:49:38 [sby/stopwatch_counter_div_7] summary: engine_0 did not produce any traces
SBY  8:49:38 [sby/stopwatch_counter_div_7] summary: successful proof by k-induction.
SBY  8:49:38 [sby/stopwatch_counter_div_7] DONE (PASS, rc=0)
SBY  8:49:38 [sby/stopwatch_counter_div_3] engine_0.basecase: ##   0:00:00  Status: passed
SBY  8:49:38 [sby/stopwatch_counter_div_3] engine_0.basecase: finished (returncode=0)
SBY  8:49:38 [sby/stopwatch_counter_div_3] engine_0.basecase: Status returned by engine for basecase: pass
SBY  8:49:38 [sby/stopwatch_counter_div_3] summary: Elapsed clock time [H:MM:SS (secs)]: 0:00:00 (0)
SBY  8:49:38 [sby/stopwatch_counter_div_3] summary: Elapsed process time [H:MM:SS (secs)]: 0:00:02 (2)
SBY  8:49:38 [sby/stopwatch_counter_div_3] summary: engine_0 (smtbmc z3) returned pass for basecase
SBY  8:49:38 [sby/stopwatch_counter_div_3] summary: engine_0 (smtbmc z3) returned pass for induction
SBY  8:49:38 [sby/stopwatch_counter_div_3] summary: engine_0 did not produce any traces
SBY  8:49:38 [sby/stopwatch_counter_div_3] summary: successful proof by k-induction.
SBY  8:49:38 [sby/stopwatch_counter_div_3] DONE (PASS, rc=0)
SBY  8:49:38 [sby/stopwatch_counter_div_8] engine_0.induction: ##   0:00:00  Temporal induction successful.
SBY  8:49:38 [sby/stopwatch_counter_div_8] engine_0.induction: ##   0:00:00  Status: passed
SBY  8:49:38 [sby/stopwatch_counter_div_8] engine_0.induction: finished (returncode=0)
SBY  8:49:38 [sby/stopwatch_counter_div_8] engine_0.induction: Status returned by engine for induction: pass
SBY  8:49:38 [sby/stopwatch_counter_div_8] summary: Elapsed clock time [H:MM:SS (secs)]: 0:00:00 (0)
SBY  8:49:38 [sby/stopwatch_counter_div_8] summary: Elapsed process time [H:MM:SS (secs)]: 0:00:02 (2)
SBY  8:49:38 [sby/stopwatch_counter_div_8] summary: engine_0 (smtbmc z3) returned pass for basecase
SBY  8:49:38 [sby/stopwatch_counter_div_8] summary: engine_0 (smtbmc z3) returned pass for induction
SBY  8:49:38 [sby/stopwatch_counter_div_8] summary: engine_0 did not produce any traces
SBY  8:49:38 [sby/stopwatch_counter_div_8] summary: successful proof by k-induction.
SBY  8:49:38 [sby/stopwatch_counter_div_8] DONE (PASS, rc=0)
root@0b9c66e66b04:/workspace#
```

```
root@0b9c66e66b04:/workspace# pytest -m formal -v
================================= test session starts ==================================
platform linux -- Python 3.12.3, pytest-9.0.3, pluggy-1.7.0 -- /opt/venv/bin/python3
cachedir: .pytest_cache
metadata: {'Python': '3.12.3', 'Platform': 'Linux-7.0.12-linuxkit-x86_64-with-glibc2.39',
'Packages': {'pytest': '9.0.3', 'pluggy': '1.7.0'}, 'Plugins': {'metadata': '3.1.1', 'html
': '4.2.0'}}
rootdir: /workspace
configfile: .pytest.toml
testpaths: tests
plugins: metadata-3.1.1, html-4.2.0
collected 118 items / 110 deselected / 8 selected

tests/test_formal.py::test_formal_sby[cascade_counter] PASSED                    [ 12%]
tests/test_formal.py::test_formal_sby[editable_countdown] SKIPPED (editable_co...) [ 25%]
tests/test_formal.py::test_formal_sby[snapshot_mux] SKIPPED (snapshot_mux modu...) [ 37%]
tests/test_formal.py::test_formal_sby[stopwatch_control] SKIPPED (stopwatch_co...) [ 50%]
tests/test_formal.py::test_formal_sby[stopwatch_counter] PASSED                  [ 62%]
tests/test_formal.py::test_formal_sby[up_down_counter_rst] PASSED                [ 75%]
tests/test_formal.py::test_formal_sby[user_top_brightness_wrapper] SKIPPED (us...) [ 87%]
tests/test_formal.py::test_formal_sby[user_top_timer_v1] SKIPPED (user_top_tim...) [100%]

====================== 3 passed, 5 skipped, 110 deselected in 1.11s ======================
root@0b9c66e66b04:/workspace#
```

Report 8

Explain how your implementation satisfies the first-tick timing constraint: the first centisecond increment must occur one centisecond after `enable` goes high, not sooner.

The Restartable Rate Generator (RRG) module does not have an input `reset` port to reset it's clock, it does so by taking the invert of the input `run` port: Disabling the RRG module resets it's internal count. It is tempting to wire the RRG's output `tick` port to the Cascade Counter (CC) module input `enable` port. This is incorrect as in the edge case of `tick`=HIGH and `enable`=LOW, the CC counter will tick which is not what we want.

We thus have to

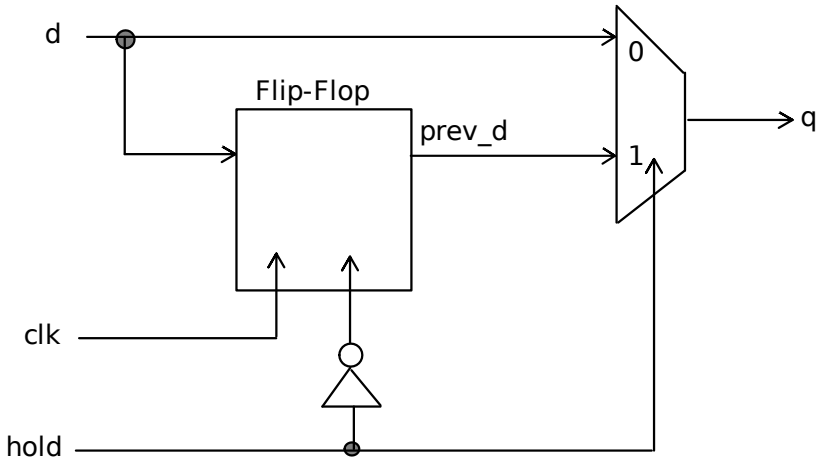
4.5. Snapshot Multiplexer

When `hold` is low, `q` equals `d` combinatorially — there is no clock edge involved; any change in `d` propagates to `q` immediately.

When `hold` is high, `q` is frozen at the value `d` held on the last rising clock edge before `hold` went high.

Report 9

Draw a diagram showing how this is implemented in hardware.



rtl > snapshot_mux.sv > SystemVerilog - Language Support > snapshot_mux

```sv
// Snapshot Multiplexer
// Output `q` takes `d` when `hold` is LOW. When `hold` is HIGH, q takes last
// known `d` during `hold`'s last LOW that crossed the rising edge.
//
// Parameters:
//    WIDTH:  - Bit width of input `d` and output `q`.
//
// Ports:
//    clk:            - Clock input
//    hold:           - Input Signal representing the Mux Switch's `sel`.
//    [WIDTH-1:0] d:  - Input Signal, D-FlipFlop `d` input
//    [WIDTH-1:0] q:  - Output Signal, D-FlipFlop `q` output
//
// Local Variables:
//    [WIDTH-1:0] prev_d: - Intermediary Signal, stores `d` at `hold`'s last
//                          LOW signal


module snapshot_mux #(
    parameter int WIDTH = 1
) (
    input logic clk,
    input logic hold,
    input logic [WIDTH-1:0] d,
    output logic [WIDTH-1:0] q
);

  logic [WIDTH-1:0] prev_d = WIDTH'(0);

  always_ff @(posedge clk) begin
    if (!hold) prev_d <= d;
  end

  always_comb begin
    if (!hold) q = d;
    else q = prev_d;
  end

endmodule

```

```
root@0b9c66e66b04:/workspace# verilator -Wall --lint-only -y rtl rtl/snapshot_mux.sv
root@0b9c66e66b04:/workspace# iverilog -Wall -g2012 -Y .sv -y rtl rtl/snapshot_mux.sv
root@0b9c66e66b04:/workspace#
```

```
SBY 10:11:27 [sby/snapshot_mux_width_1] engine_0.induction: Status returned by engine for induction: pass
SBY 10:11:27 [sby/snapshot_mux_width_1] summary: Elapsed clock time [H:MM:SS (secs)]: 0:00:00 (0)
SBY 10:11:27 [sby/snapshot_mux_width_1] summary: Elapsed process time [H:MM:SS (secs)]: 0:00:00 (0)
SBY 10:11:27 [sby/snapshot_mux_width_1] summary: engine_0 (smtbmc z3) returned pass for basecase
SBY 10:11:27 [sby/snapshot_mux_width_1] summary: engine_0 (smtbmc z3) returned pass for induction
SBY 10:11:27 [sby/snapshot_mux_width_1] summary: engine_0 did not produce any traces
SBY 10:11:27 [sby/snapshot_mux_width_1] summary: successful proof by k-induction.
SBY 10:11:27 [sby/snapshot_mux_width_1] DONE (PASS, rc=0)
SBY 10:11:27 [sby/snapshot_mux_width_20] engine_0.basecase: ##   0:00:00  Solver: z3
SBY 10:11:27 [sby/snapshot_mux_width_20] engine_0.basecase: ##   0:00:00  Checking assumptions in step 0..
SBY 10:11:27 [sby/snapshot_mux_width_20] engine_0.basecase: ##   0:00:00  Checking assertions in step 0..
SBY 10:11:27 [sby/snapshot_mux_width_20] engine_0.basecase: ##   0:00:00  Checking assumptions in step 1..
SBY 10:11:27 [sby/snapshot_mux_width_20] engine_0.basecase: ##   0:00:00  Checking assertions in step 1..
SBY 10:11:27 [sby/snapshot_mux_width_20] engine_0.basecase: ##   0:00:00  Checking assumptions in step 2..
SBY 10:11:27 [sby/snapshot_mux_width_20] engine_0.basecase: ##   0:00:00  Checking assertions in step 2..
SBY 10:11:27 [sby/snapshot_mux_width_20] engine_0.basecase: ##   0:00:00  Status: passed
SBY 10:11:27 [sby/snapshot_mux_width_20] engine_0.basecase: finished (returncode=0)
SBY 10:11:27 [sby/snapshot_mux_width_20] engine_0.basecase: Status returned by engine for basecase: pass
SBY 10:11:27 [sby/snapshot_mux_width_20] summary: Elapsed clock time [H:MM:SS (secs)]: 0:00:00 (0)
SBY 10:11:27 [sby/snapshot_mux_width_20] summary: Elapsed process time [H:MM:SS (secs)]: 0:00:00 (0)
SBY 10:11:27 [sby/snapshot_mux_width_20] summary: engine_0 (smtbmc z3) returned pass for basecase
SBY 10:11:27 [sby/snapshot_mux_width_20] summary: engine_0 (smtbmc z3) returned pass for induction
SBY 10:11:27 [sby/snapshot_mux_width_20] summary: engine_0 did not produce any traces
SBY 10:11:27 [sby/snapshot_mux_width_20] summary: successful proof by k-induction.
SBY 10:11:27 [sby/snapshot_mux_width_20] DONE (PASS, rc=0)
root@0b9c66e66b04:/workspace#
```

```
root@0b9c66e66b04:/workspace# pytest -m formal -v
================================== test session starts ==================================
platform linux -- Python 3.12.3, pytest-9.0.3, pluggy-1.7.0 -- /opt/venv/bin/python3
cachedir: .pytest_cache
metadata: {'Python': '3.12.3', 'Platform': 'Linux-7.0.12-linuxkit-x86_64-with-glibc2.39', '
Packages': {'pytest': '9.0.3', 'pluggy': '1.7.0'}, 'Plugins': {'metadata': '3.1.1', 'html':
 '4.2.0'}}
rootdir: /workspace
configfile: .pytest.toml
testpaths: tests
plugins: metadata-3.1.1, html-4.2.0
collected 120 items / 112 deselected / 8 selected

tests/test_formal.py::test_formal_sby[cascade_counter] PASSED                    [ 12%]
tests/test_formal.py::test_formal_sby[editable_countdown] SKIPPED (editable_cou...) [ 25%]
tests/test_formal.py::test_formal_sby[snapshot_mux] PASSED                       [ 37%]
tests/test_formal.py::test_formal_sby[stopwatch_control] SKIPPED (stopwatch_con...) [ 50%]
tests/test_formal.py::test_formal_sby[stopwatch_counter] PASSED                  [ 62%]
tests/test_formal.py::test_formal_sby[up_down_counter_rst] PASSED                [ 75%]
tests/test_formal.py::test_formal_sby[user_top_brightness_wrapper] SKIPPED (use...) [ 87%]
tests/test_formal.py::test_formal_sby[user_top_timer_v1] SKIPPED (user_top_time...) [100%]

================== 4 passed, 4 skipped, 112 deselected in 1.44s ==================
root@0b9c66e66b04:/workspace#
```

4.6. Stopwatch Control

It is assumed that the instantiating module will use rising-edge detectors, so the two inputs are single-cycle pulses, each high for exactly one clock cycle when the corresponding button is pressed.

The module implements the FSM described implicitly in Section 4.1. The following additional constraints apply.

All outputs are initialised to 0.

The `counter_rst` output is high for exactly one clock cycle when asserted: as soon as `rise_lap` falls, the FSM exits the Reset state.

Report 10

Draw the FSM state diagram before implementing it. After completing the implementation, note any differences between your diagram and the final design, and reflect on what caused them.

The formal verification tool relies on `yosys` to compile the SystemVerilog code. However, `yosys` can promote `logic x=y;` into `assign x=y;`, and so may not detect that the former is an initialisation rather than a continuous assignment. An automated test bench has therefore also been provided: run `pytest`.

Hints

- Using `{counter_rst, counter_enable, lap_hold}` as the state variable makes the code easy to read and to write.
- Define the next-state logic for `counter_rst`, `counter_enable`, and `lap_hold` in three separate blocks, one per signal: two require only a single `assign` statement each, and the third requires an `always_comb` block.

Additional Oral Preparation 1

The hint above suggests encoding the FSM state directly as its output signals rather than using a separate state variable. Be prepared to explain why this works and what advantage it offers.

rtl > stopwatch_control.sv > ...

```sv
// Stopwatch Control
// When in Stopwatch mode, input buttons KEY[1:0] control the state of
// the stopwatch
//
// Parameters:
//    None
//
// Ports:
//    clk:             - Clock input
//    rise_start_stop: - Input signal from KEY[0], assumed passed in as a
//                       pulse from a Rising Edge Detector module
//    rise_lap:        - Input signal from KEY[1], same assumption as above
//    counter_rst:     - Output signal, single one-cycle pulse of RESET state
//    counter_enable:  - Output signal, `enabled` state
//    counter_lap_hold:- Output signal, 'frozen display' state
//
// Local Variables:
//    state:      - Intermediary Variable to store state for output signals
//    next_state: - Intermediary Variable to store state for next rising edge
//    ss_only:    - Combinational Logic to ensure that if both buttons are
//    lap_only:       pressed, nothing happens

`timescale 1ns / 1ps

module stopwatch_control (
    input  logic clk,
    input  logic rise_start_stop,  // Button[0]
    input  logic rise_lap,         // Button[1]
    output logic counter_rst,      // Reset Pulse
    output logic counter_enable,   // Running(1) or Stopped(0)
    output logic lap_hold          // Freeze(1) or Live(0)
);

  logic [2:0] state, next_state;
  initial state = 3'b000;

  wire ss_only = rise_start_stop && !rise_lap;
  wire lap_only = !rise_start_stop && rise_lap;

  always_ff @(posedge clk) begin
    state <= next_state;
  end

  assign next_state[1] = ss_only ? !state[1] : state[1];
  assign next_state[0] = ((counter_enable || lap_hold) && lap_only) ? !state[0] : state[0];

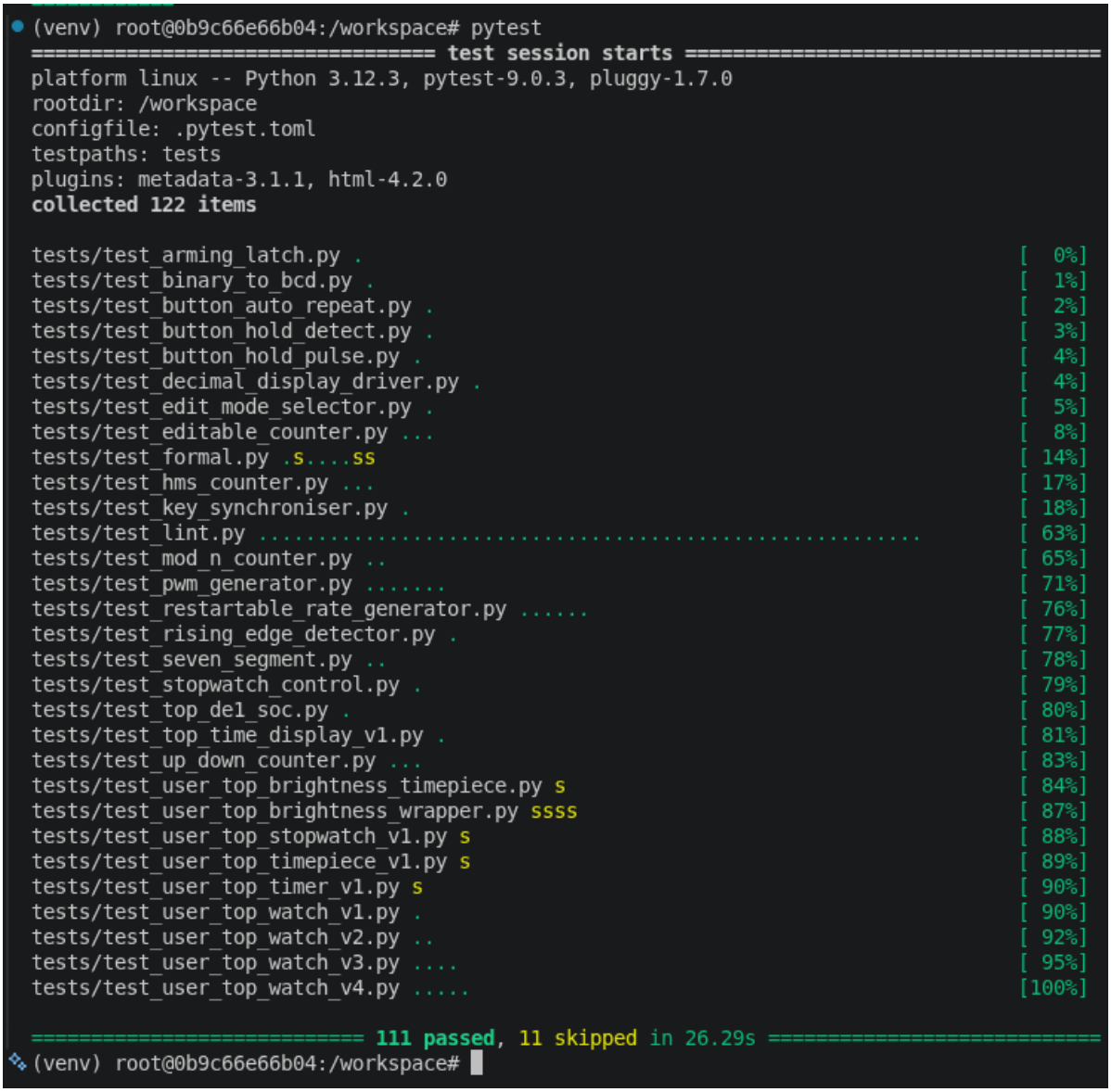
  // TODO: Fix this Latch!!!
  always_latch begin
    if (!(counter_enable || lap_hold)) begin
      next_state[2] = lap_only;
    end
  end

  assign counter_rst = state[2];
  assign counter_enable = state[1];
  assign lap_hold = state[0];

endmodule
```

```
(venv) root@0b9c66e66b04:/workspace# verilator -Wall --lint-only -y rtl rtl/stopwatch_control.sv
(venv) root@0b9c66e66b04:/workspace# iverilog -Wall -g2012 -Y .sv -y rtl rtl/stopwatch_control.sv
(venv) root@0b9c66e66b04:/workspace#
```

```
SBY  2:43:45 [sby/stopwatch_control] engine_0.induction: finished (returncode=0)
SBY  2:43:45 [sby/stopwatch_control] engine_0.induction: Status returned by engine for induction: pass
SBY  2:43:45 [sby/stopwatch_control] summary: Elapsed clock time [H:MM:SS (secs)]: 0:00:00 (0)
SBY  2:43:45 [sby/stopwatch_control] summary: Elapsed process time [H:MM:SS (secs)]: 0:00:00 (0)
SBY  2:43:45 [sby/stopwatch_control] summary: engine_0 (smtbmc z3) returned pass for basecase
SBY  2:43:45 [sby/stopwatch_control] summary: engine_0 (smtbmc z3) returned pass for induction
SBY  2:43:45 [sby/stopwatch_control] summary: engine_0 did not produce any traces
SBY  2:43:45 [sby/stopwatch_control] summary: successful proof by k-induction.
SBY  2:43:45 [sby/stopwatch_control] DONE (PASS, rc=0)
(venv) root@0b9c66e66b04:/workspace#
```


(venv) root@0b9c66e66b04:/workspace# pytest
test session starts
platform linux -- Python 3.12.3, pytest-9.0.3, pluggy-1.7.0
rootdir: /workspace
configfile: .pytest.toml
testpaths: tests
plugins: metadata-3.1.1, html-4.2.0
collected 122 items
tests/test_arming_latch.py . [0%]
tests/test_binary_to_bcd.py . [1%]
tests/test_button_auto_repeat.py . [2%]
tests/test_button_hold_detect.py . [3%]
tests/test_button_hold_pulse.py . [4%]
tests/test_decimal_display_driver.py . [4%]
tests/test_edit_mode_selector.py . [5%]
tests/test_editable_counter.py ... [8%]
tests/test_formal.py .s....ss [14%]
tests/test_hms_counter.py ... [17%]
tests/test_key_synchroniser.py . [18%]
tests/test_lint.py ..... [63%]
tests/test_mod_n_counter.py .. [65%]
tests/test_pwm_generator.py ..... [71%]
tests/test_restartable_rate_generator.py ..... [76%]
tests/test_rising_edge_detector.py . [77%]
tests/test_seven_segment.py .. [78%]
tests/test_stopwatch_control.py . [79%]
tests/test_top_de1_soc.py . [80%]
tests/test_top_time_display_v1.py . [81%]
tests/test_up_down_counter.py ... [83%]
tests/test_user_top_brightness_timepiece.py s [84%]
tests/test_user_top_brightness_wrapper.py ssss [87%]
tests/test_user_top_stopwatch_v1.py s [88%]
tests/test_user_top_timepiece_v1.py s [89%]
tests/test_user_top_timer_v1.py s [90%]
tests/test_user_top_watch_v1.py . [90%]
tests/test_user_top_watch_v2.py .. [92%]
tests/test_user_top_watch_v3.py .... [95%]
tests/test_user_top_watch_v4.py ..... [100%]
111 passed, 11 skipped in 26.29s
(venv) root@0b9c66e66b04:/workspace#

4.7. Stopwatch Integration

Task Duplicate `rtl/user_top.sv`, rename the file to `rtl/user_top_stopwatch_v1.sv`, and change the module name to `user_top_stopwatch_v1`. Then implement the stopwatch in accordance with Section 4.1, following all conventions from Assignment 1, including elimination of all Verilator linting warnings. Then run `pytest` to test it.

Dependencies Only the following modules are to be directly instantiated.

- `rising_edge_detector`
- `snapshot_mux`
- `stopwatch_control`
- `stopwatch_counter`

Note Formal verification is not used for `user_top_stopwatch_v1`. Integration modules are less amenable to it for two reasons. First, writing a complete formal specification at this level is as complex as writing the implementation itself. Second, what we want to check at the top level is that long sequences of inputs — press start, wait, press lap, press stop, check the display — produce the correct outputs; formal verification is not naturally suited to this, excelling instead at proving invariants and simple cause-and-effect properties of the form "event A implies event B on the next clock edge," which is why it works well for individual submodules. Simulation, by contrast, can directly execute arbitrary input sequences, making it the right tool for integration testing.

Additional Oral Preparation 2

Be prepared to explain why formal verification is well suited to individual submodules but less suited to integration-level testing, and why simulation is preferred at the integration level.

4.8. Programming the Board

Open the Quartus project from Assignment 1, copy your `rtl/*.sv` files into the project directory (overwriting the existing ones), then go to Project ⟩ Add/Remove Files and add any new `*.sv` files.

In `top_de1_soc.sv`, replace `user_top_watch_v4` with `user_top_stopwatch_v1` and save.

Compile and program the board.

rtl > user_top_stopwatch_v1.sv > ...

```
18  `timescale 1ns / 1ps
19
20  module user_top_stopwatch_v1 #(
21      /* verilator lint_off UNUSEDPARAM */
22      parameter int CYCLES_PER_SECOND = 50_000_000
23      /* verilator lint_on UNUSEDPARAM */
24  ) (
25      input logic clk,
26      /* verilator lint_off UNUSED */
27      input logic [3:0] button,
28      input logic [9:0] sw,
29      /* verilator lint_on UNUSED */
30      output logic [9:0] led,
31      output logic [6:0] hours_disp,
32      output logic [6:0] minutes_disp,
33      output logic [6:0] seconds_disp,
34      output logic blank_hours,
35      output logic blank_minutes,
36      output logic blank_seconds
37  );
38
39    assign led = clk ? sw : ~sw;
40
41    logic ss, lap;
42    logic counter_rst, counter_enable, lap_hold;
43    logic [6:0] minutes, minutes_hold;
44    logic [5:0] seconds, seconds_hold;
45    logic [6:0] centiseconds, centiseconds_hold;
46
47    rising_edge_detector u_red_ss (
48        .clk(clk),
49        .sig_in(button[0]),
50        .rise(ss)
51    );
52
53    rising_edge_detector u_red_lap (
54        .clk(clk),
55        .sig_in(button[1]),
56        .rise(lap)
57    );
58
59    stopwatch_control u_stopwatch_control (
60        .clk(clk),
61        .rise_start_stop(ss),
62        .rise_lap(lap),
63        .counter_rst(counter_rst),
64        .counter_enable(counter_enable),
65        .lap_hold(lap_hold)
66    );
67
68    snapshot_mux #(
69        .WIDTH(7)
70    ) u_snapshot_minutes_mux (
71        .clk(clk),
72        .hold(lap_hold),
73        .d(minutes),
74        .q(minutes_hold)
75    );
76
77    snapshot_mux #(
78        .WIDTH(6)
79    ) u_snapshot_seconds_mux (
80        .clk(clk),
81        .hold(lap_hold),
82        .d(seconds),
83        .q(seconds_hold)
84    );
85
86    snapshot_mux #(
87        .WIDTH(7)
88    ) u_snapshot_centiseconds_mux (
89        .clk(clk),
90        .hold(lap_hold),
91        .d(centiseconds),
92        .q(centiseconds_hold)
93    );
94
95    stopwatch_counter #(
96        .CYCLES_PER_SECOND(CYCLES_PER_SECOND)
97    ) u_stopwatch_counter (
98        .clk(clk),
99        .rst(counter_rst),
100       .enable(counter_enable),
101       .minutes(minutes),
102       .seconds(seconds),
103       .centiseconds(centiseconds)
104   );
105
106   assign hours_disp = (lap_hold) ? minutes_hold : minutes;
107   assign minutes_disp = (lap_hold) ? {1'b0, seconds_hold} : {1'b0, seconds};
108   assign seconds_disp = (lap_hold) ? centiseconds_hold : centiseconds;
109
110   assign blank_hours = 1'b0;
111   assign blank_minutes = 1'b0;
112   assign blank_seconds = 1'b0;
113
114 endmodule
115
```

```
(venv) root@0b9c66e66b04:/workspace# verilator -Wall --lint-only -y rtl rtl/user_top_stopwatch_v1.sv
(venv) root@0b9c66e66b04:/workspace# iverilog -Wall -g2012 -Y .sv -y rtl rtl/user_top_stopwatch_v1.sv
(venv) root@0b9c66e66b04:/workspace#
```

```
(venv) root@0b9c66e66b04:/workspace# pytest
================================= test session starts ==================================
platform linux -- Python 3.12.3, pytest-9.0.3, pluggy-1.7.0
rootdir: /workspace
configfile: .pytest.toml
testpaths: tests
plugins: metadata-3.1.1, html-4.2.0
collected 124 items

tests/test_arming_latch.py .                                                 [  0%]
tests/test_binary_to_bcd.py .                                                [  1%]
tests/test_button_auto_repeat.py .                                           [  2%]
tests/test_button_hold_detect.py .                                           [  3%]
tests/test_button_hold_pulse.py .                                            [  4%]
tests/test_decimal_display_driver.py .                                       [  4%]
tests/test_edit_mode_selector.py .                                           [  5%]
tests/test_editable_counter.py ...                                           [  8%]
tests/test_formal.py .s....ss                                                [ 14%]
tests/test_hms_counter.py ...                                                [ 16%]
tests/test_key_synchroniser.py .                                             [ 17%]
tests/test_lint.py ..........................................................  [ 64%]
tests/test_mod_n_counter.py ..                                               [ 66%]
tests/test_pwm_generator.py .......                                          [ 71%]
tests/test_restartable_rate_generator.py ......                              [ 76%]
tests/test_rising_edge_detector.py .                                         [ 77%]
tests/test_seven_segment.py ..                                               [ 79%]
tests/test_stopwatch_control.py .                                            [ 79%]
tests/test_top_de1_soc.py .                                                  [ 80%]
tests/test_top_time_display_v1.py .                                          [ 81%]
tests/test_up_down_counter.py ...                                            [ 83%]
tests/test_user_top_brightness_timepiece.py s                                [ 84%]
tests/test_user_top_brightness_wrapper.py ssss                               [ 87%]
tests/test_user_top_stopwatch_v1.py .                                        [ 88%]
tests/test_user_top_timepiece_v1.py s                                        [ 89%]
tests/test_user_top_timer_v1.py s                                            [ 90%]
tests/test_user_top_watch_v1.py .                                            [ 91%]
tests/test_user_top_watch_v2.py ..                                           [ 92%]
tests/test_user_top_watch_v3.py ....                                         [ 95%]
tests/test_user_top_watch_v4.py .....                                        [100%]

========================== 114 passed, 10 skipped in 25.66s ============================
(venv) root@0b9c66e66b04:/workspace#
```

5. Timer

With the stopwatch complete, we now design and implement a timer.

5.1. Specification

The timer uses the same time-setting interface as the settable watch from Assignment 1. The timer has three modes: set, stopped, and running.

Stopped Mode. The display shows the current timer value. Pressing `button[0]` starts the timer if the count is greater than zero; otherwise it has no effect. The first decrement occurs not sooner than one second after the button is pressed. Holding `button[3]` for at least one second enters set mode.

Set Mode. The seconds digits flash at 2 Hz with an 80% duty cycle, indicating the currently selected field. The flashing digits are incremented by `button[1]` and decremented by `button[0]`; holding either button for more than half a second repeats at 10 Hz. A second press of `button[3]` moves to the minutes digits, a third to the hours digits, and a fourth exits set mode and returns to stopped mode with the newly set time.

Running Mode. The display counts down once per second. Pressing `button[0]` pauses the timer. When the count reaches zero, the timer stops automatically and the display holds at zero. Set mode cannot be entered while the timer is running.

5.2. Design Approach

The time-setting procedure is identical to that in Assignment 1, so the same modules can be instantiated. In fact, apart from three small differences, the timer is equivalent to a settable watch: time ticks down rather than up; time stops at zero instead of wrapping; and time can be paused and resumed.

We consider each difference in turn.

Time Ticks Down. The underlying counters simply need their direction reversed.

Time Stops at Zero. A combinational check determines whether the current count is all zeros; this produces a signal that qualifies the counter enable, preventing any further decrement once zero is reached.

Pausing and Resuming. The tick signal that drives the counters is qualified by a running/stopped flag. When the timer is paused, the tick is suppressed and the counters hold their value.

Since the required changes are small, the overall design closely mirrors that of the settable watch from Assignment 1. Try to derive it independently; if you get stuck, the settable watch implementation is there as a reference.

5.3. Editable Countdown with Borrow Out

Report 11

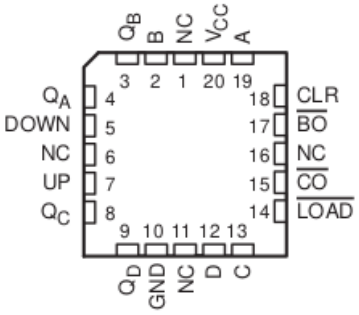
Find the datasheet for the 74LS193 synchronous 4-bit up/down counter. Using the schematic diagram in the datasheet, explain how the borrow output is generated and why it behaves as it does.

description

The 'ALS193A are synchronous, reversible, 4-bit up/down binary counters. Synchronous counting operation is provided by having all flip-flops clocked simultaneously so that the outputs change coincident with each other when instructed by the steering logic. This mode of operation eliminates the output counting spikes normally associated with asynchronous (ripple-clock) counters.

The outputs of the four flip-flops are triggered on a low-to-high-level transition of either count/clock (UP or DOWN) input. The direction of the count is determined by which count input is pulsed while the other count input is high.

SN54ALS193A . . . FK PACKAGE
(TOP VIEW)



NC – No internal connection

All four counters are fully programmable; that is, each output may be preset to either level by placing a low on the load ($\overline{LOAD}$) input and entering the desired data at the data inputs. The output changes to agree with the data inputs independently of the count pulses. This feature allows the counters to be used as modulo-N dividers by simply modifying the count length with the preset inputs.

A high level applied to the clear (CLR) input forces all outputs to the low level. The clear function is independent of the count and $\overline{LOAD}$ inputs. The UP, DOWN, and $\overline{LOAD}$ inputs are buffered to lower the drive requirement, which significantly reduces the loading on, or current required by, clock drivers, etc., for long parallel words.

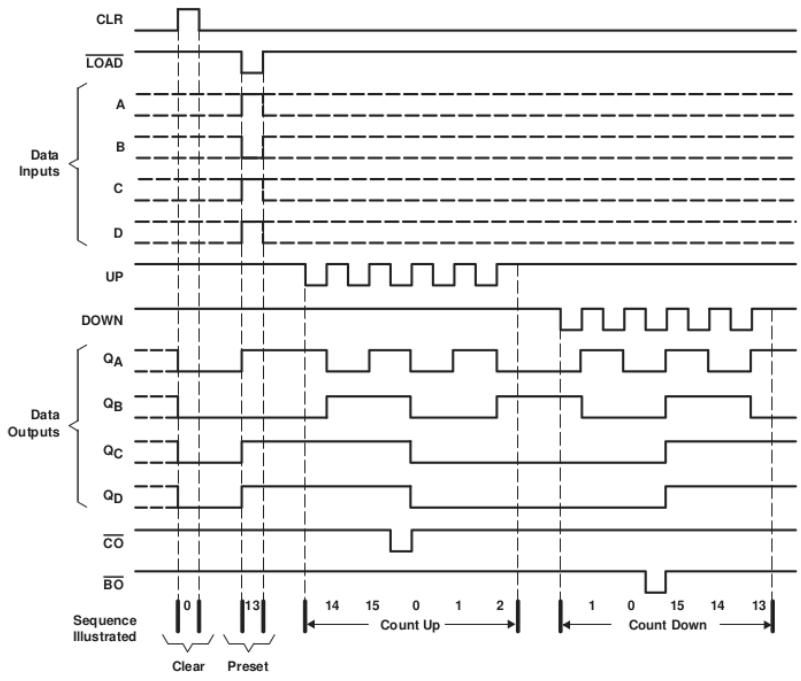
These counters are designed to be cascaded without the need for external circuitry. The borrow ($\overline{BO}$) output produces a low-level pulse while the count is zero (all Q outputs low) and the DOWN input is low. Similarily, the carry ($\overline{CO}$) output produces a low-level pulse while the count is 9 or 15 (all Q outputs high) and the UP input is low. The counters can then be easily cascaded by feeding $\overline{BO}$ and $\overline{CO}$ to the count-down and count-up inputs, respectively, of the succeeding counter.

The SN54ALS193A is characterized for operation over the full military temperature range of −55°C to 125°C. The SN74ALS193A is characterized for operation from 0°C to 70°C.

typical clear, load, and count sequence

the following sequence is illustrated below:

1. Clear outputs to zero
2. Load (preset) to binary 13
3. Count up to 14, 15 (carry), 0, 1, and 2
4. Count down to 1, 0 (borrow), 15, 14, and 13



NOTES: A. Clear overrides load, data, and count inputs.
B. When counting up, count-down input must be high; when counting down, count-up input must be high.

Instead of using CLOCK_50 directly for pulses, pair with UP/DOWN states as the 'CLK' for both directions

The 4-bit counter can be loaded initially with a 4-bit value

If UP is counting and the counter reaches (Moore) state 4'b1111, an output CARRY signal is generated

Likewise, when DOWN is counting and counter reaches (Moore) state 4'b0000, an output BORROW signal is generated

This module behaves like `editable_counter` from Assignment 1, with one change and two additions: `count` decrements on each `tick` pulse (wrapping from 0 to `MAX`); `clr` sets `count` to 0 on the next rising clock edge, taking priority over all other inputs; and `borrow_out` is a new combinational output mirroring the borrow output of the 74LS193, except that it is also low when `edit_mode` or `clr` is high. The `count` output is initialised to 0.

Dependency Only the following module is to be directly instantiated.

- `up_down_counter_rst`

rtl > editable_countdown.sv > ...

```
// Editable Countdown
// Same as the Editable Counter module but with `clr`, counts down and extra
// output signal `borrow_out`
//
// Parameters:
//    MAX
//    WIDTH
//
// Ports:
//    clk:              - Clock input
//    clr:              - Asynchronous reset
//    tick:             - Input signal to pulse the `tick_event` when not in
//                        `edit_mode`
//    edit_mode:        - Input signal to represent State whether module is
//                      - in edit mode
//    inc:              - Input signal to pulse the `inc_event` in `edit_mode`
//    dec:              - Input signal to pulse the `dec_event` in `edit_mode`
//    [WIDTH-1:0] count:- Output signal to count a number between 0 to N-1
//    borrow_out:       - Output signal to pulse rollovers
//
// Local Variables:
//    inc_event:  - Signal representing incrementing STATE when
//                  `inc` is HIGH AND `dec` is LOW while in `edit_mode`
//    dec_event:  - Signal representing decrementing STATE when
//                  `dec` is HIGH AND `inc` is LOW while in `edit_mode`
//    tick_event: - Signal representing NOT in `edit_mode` running according
//                  to `tick` pulses
//    up:         - Direction of counting: HIGH = upwards, LOW = downwards
//    enable:     - Enable the Up_Down_Counter when correct conditions are met

`timescale 1ns / 1ps

module editable_countdown #(
    parameter int MAX   = 59,
    parameter int WIDTH = 6
) (
    input logic clk,
    input logic clr,
    input logic tick,
    input logic edit_mode,
    input logic inc,
    input logic dec,
    output logic [WIDTH-1:0] count,
    output logic borrow_out
);

  logic enable;
  logic up;
  up_down_counter_rst #(
      .MAX  (MAX),
      .WIDTH(WIDTH)
  ) u_up_down_counter_rst (
      .clk(clk),
      .rst(clr),
      .enable(enable),
      .up(up),
      .count(count)
  );

  wire inc_event = edit_mode && inc && !dec && !clr;
  wire dec_event = edit_mode && dec && !inc && !clr;
  wire tick_event = !edit_mode && tick && !clr;

  always_comb begin
    if (edit_mode) up = inc_event || !dec_event;
    else up = !tick_event;
  end

  assign enable = tick_event || (dec_event || inc_event);

  assign borrow_out = tick_event && (count == 0);

endmodule
```

```
root@0b9c66e66b04:/workspace# verilator -Wall --lint-only -y rtl rtl/editable_countdown.sv
root@0b9c66e66b04:/workspace# iverilog -Wall -g2012 -Y .sv -y rtl rtl/editable_countdown.sv
root@0b9c66e66b04:/workspace#
```

```
SBY  7:32:21 [sby/editable_countdown_max_4] summary: Elapsed clock time [H:MM:SS (secs)]: 0:00:00 (0)
SBY  7:32:21 [sby/editable_countdown_max_4] summary: Elapsed process time [H:MM:SS (secs)]: 0:00:00 (0)
SBY  7:32:21 [sby/editable_countdown_max_4] summary: engine_0 (smtbmc z3) returned pass for basecase
SBY  7:32:21 [sby/editable_countdown_max_4] summary: engine_0 (smtbmc z3) returned pass for induction
SBY  7:32:21 [sby/editable_countdown_max_4] summary: engine_0 did not produce any traces
SBY  7:32:21 [sby/editable_countdown_max_4] summary: successful proof by k-induction.
SBY  7:32:21 [sby/editable_countdown_max_4] DONE (PASS, rc=0)
SBY  7:32:21 [sby/editable_countdown_max_59] summary: Elapsed clock time [H:MM:SS (secs)]: 0:00:00 (0)
SBY  7:32:21 [sby/editable_countdown_max_59] summary: Elapsed process time [H:MM:SS (secs)]: 0:00:00 (0)
SBY  7:32:21 [sby/editable_countdown_max_59] summary: engine_0 (smtbmc z3) returned pass for basecase
SBY  7:32:21 [sby/editable_countdown_max_59] summary: engine_0 (smtbmc z3) returned pass for induction
SBY  7:32:21 [sby/editable_countdown_max_59] summary: engine_0 did not produce any traces
SBY  7:32:21 [sby/editable_countdown_max_59] summary: successful proof by k-induction.
SBY  7:32:21 [sby/editable_countdown_max_59] DONE (PASS, rc=0)
SBY  7:32:21 [sby/editable_countdown_max_15] engine_0.induction: ##   0:00:00  Solver: z3
SBY  7:32:21 [sby/editable_countdown_max_15] engine_0.induction: ##   0:00:00  Trying induction in step 3..
SBY  7:32:21 [sby/editable_countdown_max_15] engine_0.induction: ##   0:00:00  Trying induction in step 2..
SBY  7:32:21 [sby/editable_countdown_max_15] engine_0.induction: ##   0:00:00  Trying induction in step 1..
SBY  7:32:21 [sby/editable_countdown_max_15] engine_0.induction: ##   0:00:00  Temporal induction successful.
SBY  7:32:21 [sby/editable_countdown_max_15] engine_0.induction: ##   0:00:00  Status: passed
SBY  7:32:21 [sby/editable_countdown_max_15] engine_0.induction: finished (returncode=0)
SBY  7:32:21 [sby/editable_countdown_max_15] engine_0.induction: Status returned by engine for induction: pass
SBY  7:32:21 [sby/editable_countdown_max_15] summary: Elapsed clock time [H:MM:SS (secs)]: 0:00:00 (0)
SBY  7:32:21 [sby/editable_countdown_max_15] summary: Elapsed process time [H:MM:SS (secs)]: 0:00:01 (1)
SBY  7:32:21 [sby/editable_countdown_max_15] summary: engine_0 (smtbmc z3) returned pass for basecase
SBY  7:32:21 [sby/editable_countdown_max_15] summary: engine_0 (smtbmc z3) returned pass for induction
SBY  7:32:21 [sby/editable_countdown_max_15] summary: engine_0 did not produce any traces
SBY  7:32:21 [sby/editable_countdown_max_15] summary: successful proof by k-induction.
SBY  7:32:21 [sby/editable_countdown_max_15] DONE (PASS, rc=0)
root@0b9c66e66b04:/workspace#
```

```
root@0b9c66e66b04:/workspace# pytest
========================================= test session starts ==========================================
platform linux -- Python 3.12.3, pytest-9.0.3, pluggy-1.7.0
rootdir: /workspace
configfile: .pytest.toml
testpaths: tests
plugins: metadata-3.1.1, html-4.2.0
collected 126 items

tests/test_arming_latch.py .                                                                  [  0%]
tests/test_binary_to_bcd.py .                                                                 [  1%]
tests/test_button_auto_repeat.py .                                                            [  2%]
tests/test_button_hold_detect.py .                                                            [  3%]
tests/test_button_hold_pulse.py .                                                             [  3%]
tests/test_decimal_display_driver.py .                                                        [  4%]
tests/test_edit_mode_selector.py .                                                            [  5%]
tests/test_editable_counter.py ...                                                            [  7%]
tests/test_formal.py ......ss                                                                 [ 14%]
tests/test_hms_counter.py ...                                                                 [ 16%]
tests/test_key_synchroniser.py .                                                              [ 17%]
tests/test_lint.py ............................................................               [ 65%]
tests/test_mod_n_counter.py ..                                                                [ 66%]
tests/test_pwm_generator.py .......                                                           [ 72%]
tests/test_restartable_rate_generator.py ......                                               [ 76%]
tests/test_rising_edge_detector.py .                                                          [ 77%]
tests/test_seven_segment.py ..                                                                [ 79%]
tests/test_stopwatch_control.py .                                                             [ 80%]
tests/test_top_de1_soc.py .                                                                   [ 80%]
tests/test_top_time_display_v1.py .                                                           [ 81%]
tests/test_up_down_counter.py ...                                                             [ 84%]
tests/test_user_top_brightness_timepiece.py s                                                 [ 84%]
tests/test_user_top_brightness_wrapper.py ssss                                                [ 88%]
tests/test_user_top_stopwatch_v1.py .                                                         [ 88%]
tests/test_user_top_timepiece_v1.py s                                                         [ 89%]
tests/test_user_top_timer_v1.py s                                                             [ 90%]
tests/test_user_top_watch_v1.py .                                                             [ 91%]
tests/test_user_top_watch_v2.py ..                                                            [ 92%]
tests/test_user_top_watch_v3.py ....                                                          [ 96%]
tests/test_user_top_watch_v4.py .....                                                         [100%]

================================ 117 passed, 9 skipped in 26.53s ================================
root@0b9c66e66b04:/workspace#
```

5.4. Timer Integration

Task Implement `user_top_timer_v1` in `rtl/user_top_timer_v1.sv` in accordance with Section 5.1, with no Verilator linting warnings. Develop and test one feature at a time using test benches you write yourself.

To facilitate formal verification, add two ports to your module: